

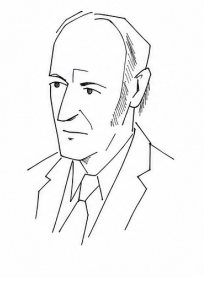



Long Division and the End of Certainty

How an elementary procedure leads to the absolute limits of mathematics

Timothy Trujillo



Contents

The next digit **v**

1 Long division at the desk **1**

2 What the digits tell us **13**

3 Drawers and games **23**

4 Instructions about instructions **35**

5 Proof on paper **45**

6 A world for the words **59**

7 Long division again **67**

Coda: A new decimal in an old world **77**

A The arithmetic sheets **81**

B A proof with the work left in **95**

Sources and Further Reading **101**

The next digit

One third fits on a scrap of paper. Its decimal expansion would use up all the paper in the world and still be asking for more. The fraction 1/3 is finished as soon as we lift the pencil; the instruction to write its digits has scarcely begun. We resolve this inconvenience with three dots:

$$\frac{1}{3} = 0.333\ldots$$

Three marks excuse us from making infinitely many more. It is an excellent arrangement, provided we know what we have been excused from doing.

Here we do. Another three belongs after the last one we wrote. Long division supplies it, along with a remainder that asks us to do exactly the same thing again. Somewhere in childhood we became capable of setting this work in motion. We were generally asked to stop after a few decimal places, which was considerate of the teacher. The calculation itself offers no such concession.

A decimal beginning, however long, can promise less than it seems. After a hundred threes a four might appear. It would no longer be one third, but it would be a perfectly respectable decimal. The marks already on the paper cannot object. Our certainty about the next three comes from the division we are doing, from the reason for the marks. This small distinction—between what has been written and what governs its continuation—will keep widening.

We will begin at the desk, with a human being employed to calculate. Following the pencil through an ordinary division will reveal how much a few instructions can accomplish. We will use instructions to write a decimal that escapes every fraction, and then share the pencil with an opponent who would prefer a different outcome. The game opens onto René Baire's mathematics: there is room inside every finite beginning for more than either player can see.

Alonzo Church and three of his students, Alan Turing, Stephen Kleene, and Leon Henkin, will enter as our demands grow. We want digits, then assurances that the digits will arrive, then proofs of those assurances. Directions that can be followed without understanding turn out to be capable of making understanding very uncomfortable. A proof about a calculation can itself become part of the calculation, with consequences for the proof.

Eventually we will return to long division with two programs and a surprise. Keep the programs fixed; choose the decimal you would like their quotient to follow. Even a decimal beyond the reach of ordinary computation can be approached in this way, as far along its digits as we can ask—in a suitable mathematical world. That last qualification will take some earning. An answer already reached by an ordinary calculation stays put. What changes is our account of a calculation that we can never finish watching.

For now there is a pencil, a sheet, and a division to be done. The infinite part can wait. It is rather good at that.

Chapter 1

Long division at the desk

The computer has gone home. On the desk are a pencil, several pages of figures, and a calculation waiting for the next shift. This was once an ordinary arrangement: a computer was a person employed to calculate. At the Langley laboratory in 1935, women calculated from experimental data, plotted results, and checked the work that went back to engineers. They also developed methods. The job required judgment, care, and a person who could sometimes notice that the instructions were wrong.[1]

We are going to take that person's chair, with one peculiar condition. The calculation must be able to continue even when the person taking over knows nothing about its purpose. Whatever understanding is needed to choose the next action must somehow be left on the desk. Turing began his investigation of computation with a person in just this situation: following definite rules, making marks, attending to one little part of the work at a time. Our first task for this person is long division.

Here is the work so far:

$$
\begin{array}{r}
15 \\
8\overline{)123} \\
8 \\
\hline
43 \\
40 \\
\hline
3
\end{array}
$$

Eight goes into twelve once, leaving four. Bring down the three and we have forty-three; five eights leave three. The answer is fifteen, with three left over. There is nothing unusual on the page

[1] NASA's historical account, "When the Computer Wore a Skirt: Langley's Computers, 1935–1970," describes the group. In 1935 the laboratory belonged to NACA.

yet, except that we intend to be rather nosy about what happens next.

Put the decimal point after fifteen. Writing the original number as 123.000 . . . gives us zeros to bring down. The first joins the remainder three, making thirty. An experienced worker sees that the next answer digit is three. Suppose we leave that little flash of recognition out of the instructions. The paper can find the digit for us:

$$30 \longrightarrow 22 \longrightarrow 14 \longrightarrow 6.$$

Take away eight, and then eight again, and then another eight. A fourth eight will not fit. Three subtractions succeeded, so three belongs after the decimal point; six remains for the following round.

With eight, this feels like taking the stairs to a place we could reach in one step. A divisor of 237 is more persuasive. How many copies fit into 1538? Successive subtractions leave $1301, 1064, 827, 590, 353, 116$: six copies fit, and a seventh does not. The little flash of recognition has become scratch work. It is slower, perhaps, but another person can pick it up where we left it.

Beside our own scratch work, reserve a square for the count. It starts at zero, becomes one after the first successful subtraction, then two, then three. The failure must leave it at three. Ordinary subtraction would let us continue $6 - 8$ into negative numbers and write -2. For division, the useful news is that another eight does not fit. We need two ways out of the subtraction: one carrying a new remainder, the other sending us back to the last remainder we accepted.

That decision will eventually have to be made by marks on the paper. To see how much a few marks can do, take the addition $587 + 468$. It contains the same small acts—look at a column, write below it, remember something for the column to its left—without the failed trial getting in the way.

$$\begin{array}{cccc} & {\scriptstyle 1} & {\scriptstyle 1} & {\scriptstyle 1} \\ & 5 & 8 & 7 \\ + & 4 & 6 & 8 \\ \hline 1 & 0 & 5 & 5 \end{array}$$

Seven and eight make fifteen. Five goes under the line and one goes above the next column. There it waits for us. Eight and six, with that one, make fifteen again. Another five, another carried one. Five and four and one make ten, giving the final zero and one of 1055. The worker has remembered the rules of addition. The page has remembered which column owes us a one.

Give each digit a square of graph paper and rest a finger in the carry row above the units. The two digits lie one and two squares below it; the answer belongs three squares below. The same arrangement will hold when the finger travels left. Here are the first two moments, just before their columns are worked:

	5	8	7
	4	6	8

Units. Blank, 7, 8.
Write 5 below; put 1 above the next column. Move left.

		1	
	5	8	7
	4	6	8
			5

Tens. 1, 8, 6.
Write 5 below; put 1 above the next column. Move left.

The second picture is what obeying the first picture's card leaves behind. We have written a five and a one, and moved our attention. Everything else stayed where it was. Do the next column and the attention arrives above the five. Do that one and it arrives over two empty squares, with a carried one still waiting:

	1	1	
	5	8	7
	4	6	8
		5	5

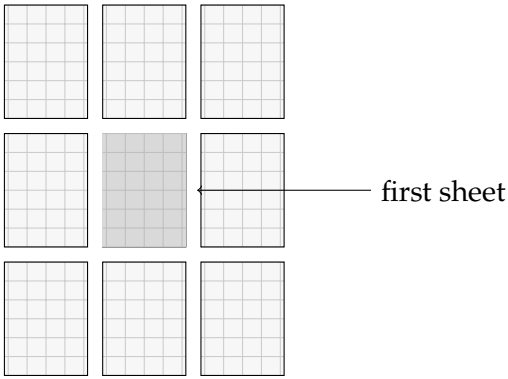
Hundreds. 1, 5, 4.
Write 0 below; put 1 above the next column. Move left.

1	1	1	
	5	8	7
	4	6	8
	0	5	5

Thousands. 1, blank, blank.
Write the remaining 1 below. The addition is finished.

The empty squares have supplied information too: both numbers have ended. We put the carried one below the line and finish. In 12 + 3, one number runs out before the other. Above the blank to the left of the three sits the one from twelve; that column still has work to do. The instruction must distinguish one blank from two.

Now suppose the sheet ends immediately to the left of our five. The final carried one has nowhere to sit. A second sheet, with its squares lined up against the first, supplies the missing column. We have changed the amount of paper, without changing what "one square left" means. The same repair works at the other edges. Sheets can meet along sides and around corners, like this:



The small gaps let us see the individual sheets; on the desk the squares join. When this larger patch runs out, more sheets can be attached. We allow the supply to continue for as long as the work requires. At any particular moment there are only finitely many marks and sheets, but we have declined to settle in advance where the last edge will be.

The addition has used four particular cards. To cover any addition, we need all the possible column facts. Put them into two tables, one for a column with a carry arriving and one without. The upper digit picks the row; the lower picks the column. Each entry says what to write below and whether to leave a carry on the left. Blank has its own row and column. Ten digits and blank give eleven observations in each number square: $11 \times 11 = 121$ entries per table, 242 altogether. The tables have room for a million-digit

addition without becoming million-digit tables. Its columns will keep asking the same small collection of questions.

Take those questions back to $30 - 8$, the first subtraction still waiting in our division. Zero cannot supply eight, so we borrow a ten. Ten minus eight gives two; the three tens have become two tens:

$$\begin{array}{rcl} & {\scriptstyle 2} & {\scriptstyle 10} \\ & 3 & 0 \\ - & & 8 \\ \hline & 2 & 2 \end{array} \qquad 30 - 8 = 22.$$

There is another way to leave the borrowing reminder. Keep the three unchanged and put a one above its column: "subtract one extra here; the column on the right has borrowed it." We can then read the original numbers throughout the trial. Subtraction gets two tables of its own, depending on whether a borrow is arriving. Our two columns use these entries:

upper	lower	borrow in	write	borrow left
0	8	blank	2	1
3	blank	1	2	blank

At the next step left, both numbers have ended and nothing is owed. The subtraction succeeds. Twenty-two can become our next remainder.

In the last trial, $6 - 8$, the units column borrows a ten and temporarily writes eight. Move left: two blanks, and the borrowed ten has nowhere to come from. This entry takes the other exit. The eight we wrote is scratch work from a failed trial, not a strangely optimistic answer to $6 - 8$. The worker need not recognize failure by looking at the whole subtraction. It is already there in the two blanks and the unpaid borrow.

For $8 - 8$, the units entry writes zero without borrowing. The next two blanks report success. One eight fitted exactly, and we must count it before trying $0 - 8$ and meeting the refusal. Both tables include their blank cases, giving the same 242 possibilities as addition. Long rows only repeat these local choices. In $1000 - 1$, each zero passes the borrow left; the one pays it, and the answer row says 0999. We will tidy that extra zero shortly.

Our arithmetic is ready to travel, but the places are becoming a nuisance. One finger is holding the answer square after 15.. Others want to follow the two numbers and the trial answer. The instruction "put it back there" is beginning to ask a lot of *there*. Give that square an address. Call the original working surface page zero and the answer row row zero. Put the first answer digit in column zero, with column numbers increasing to the right. Then the square after 15. has address page zero, column three, row zero:

$$\texttt{OUT}: \quad (0, 3, 0).$$

Write it on a card labeled `OUT`. The label says what the card is for; its three numbers say where to put the pencil. Rows increase upward, and negative row or column numbers continue below or to the left of our chosen starting square. A *place card* can hold a place while its owner's attention goes elsewhere.

The joined sheets make one expandable page. For a subtraction trial, we want a different page, where a mistake cannot overwrite the remainder we still need. Call the first scratch surface page one, the next page two, and so on. Keep the same column and row numbers on each. The first trial leaves twenty-two on page one; the next leaves fourteen on page two. Arrange their answer units at column one, row zero. The one of fourteen is then at $(2, 0, 0)$, the four at $(2, 1, 0)$. A new trial can copy them to page three.

Here is the whole copying journey. A card `A` begins at the one on page two; `B` begins at $(3, 0, 2)$, an empty square on page three. Read one at `A`, write one at `B`, move both places right. Read four, write four, move both right again. `A` now finds blank, which ends the copy. Fourteen has arrived on the new page without disappearing from the old one. Ten branches cover the possible digits; an eleventh covers blank. The one-branch says to write one, the four-branch to write four, and every digit branch makes the same two moves before looking again.

We also need to put eight under the four, not under the one. Start at the first copied digit, walk right to blank, and step back once. That is the units square. Save it, then move down one row and write eight. For a longer divisor, begin with its units digit too

and copy leftward from there. Lining up the ends aligns all the columns, even when the numbers have different lengths.

These directions are acquiring a compact written form. If `C` holds an answer place, `Write(C,5)` puts a five there. `Move(C,0,-1,0)` moves that place one column left: zero change of page, minus one column, zero change of row. The five does not move. `Place(U,C)` copies the address onto another card, `U`, so that `C` can go away and later return by following `Place(C,U)`. Saving a place copies its address, not the digit at that address.

For the arithmetic choice, `If(A,7)` asks whether the upper square contains seven. A successful test can send us to another labeled line by `Goto`. Here is the seven-plus-eight branch from our addition, with `A` and `B` at the two digits, `C` below them at the answer, and `D` in the blank carry square above:

```
pair78:  If(A,7) Goto check8
         Goto other-pairs
check8:  If(B,8) Goto write5
         Goto other-pairs
write5:  Write(C,5)
         Move(D,0,-1,0)
         Write(D,1)
         Move(A,0,-1,0)
         Move(B,0,-1,0)
         Move(C,0,-1,0)
         Goto next-column
```

Follow it with the first grid picture in view. Seven passes the first test and eight passes the second. We write five, move the carry place left and write one, then move the two number places and the answer place left. We have reached the second picture. If either test failed, the next line would send us to the other pairs instead. At `next-column`, the carried one selects the addition table with an incoming carry. A label is simply a place to resume reading the instructions.

This fragment is one doorway into a larger sheet. Its other doorways handle the other entries, including blanks and the finishing cases. The subtraction tables can be written out in the same manner. The complete sheets live in Appendix A; they are long because a

person can know seven plus eight without being told what to do when the seven turns out to be a six. Our worker must be told.

On page three, $14 - 8$ has meanwhile succeeded. The two columns have left 06. Erase the zero and keep the six. That seems simple until we try $1234 - 1234$, whose columns leave 0000. Erase every zero and we lose the answer altogether. As we set up this subtraction, let `U` save the units square of the answer. After success, the working card sits just left of the four zeros. One move right brings it to the first zero. This is not the saved units square, so we erase the zero and move on. The second and third zeros go the same way. At the fourth, the working card reaches the place saved on `U`. The walk ends with that zero still on the paper:

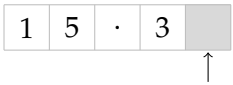
$$000\underline{0} \longrightarrow 00\underline{0} \longrightarrow 0\underline{0} \longrightarrow \underline{0}.$$

The display omits the blank squares left behind. The underlined zero stays in its original square. With 098, the first zero goes, but the nine ends the walk before it reaches the saved units square. With 1002, the first digit ends the walk immediately; neither interior zero is touched. In each case the walk checks whether it has reached the saved last digit or a nonzero digit before erasing anything. The place check has a short written form: `IfSame(A,U) Goto finished`. It compares the addresses on the two cards. Our four-zero example shows why this test comes before the erasure.

Now our four trials can keep their separate pages. A card identifies the last accepted remainder, and it changes only after success:

page	trial	count	remainder kept
1	$30 - 8 = 22$	1	22 on page 1
2	$22 - 8 = 14$	2	14 on page 2
3	$14 - 8 = 6$	3	6 on page 3
4	$6 - 8$: no	3	6 on page 3

The failed trial has left writing on page four. It can stay there; we have no reason to trust it with the next round. The count is still three, the remainder card still points to six on page three, and `OUT` still holds $(0, 3, 0)$. Copy three to that waiting square and move `OUT` one place right:



OUT: (0, 4, 0)

Bring down zero beside the accepted six. Sixty takes us back to the trial instructions, with the counting square reset to zero. Seven successful subtractions leave four. Write seven, bring down zero to make forty, and five subtractions leave zero. Our division has reached 15.375:

$$
\begin{array}{r}
15.375 \\
8\overline{)\,123.000} \\
\underline{8} \\
43 \\
\underline{40} \\
3.0 \\
\underline{2.4} \\
60 \\
\underline{56} \\
40 \\
\underline{40} \\
0
\end{array}
$$

Another round starts with zero, refuses its first subtraction and prints its unchanged count, zero. So does the next. The directions continue 15.375000 . . ., long after a person would usually put the pencil down.

The return to earlier instructions is a *loop*. It returns the worker to a line on the instruction sheet, while the answer keeps growing on the work page. A fixed piece of writing can therefore direct work that has no last step. What changes from one trip to the next is waiting on the desk: the remainder, the count, the place at which the next digit belongs.

We entered the division partway through, but the same route supplies its beginning. Start with remainder zero and bring down the supplied digits of 123 one at a time. The first gives one; no eight fits. Ignore that initial answer zero. Bring down two, making

twelve; one eight fits. Bring down three beside the remaining four, making forty-three; five eights fit. The supplied digits have ended, so write the point and begin bringing down zeros. When the integer answer has not begun at all, as in 2/8, put down 0.. After the point, every zero counts as a digit and must be kept.

The single counting square will never need ten. Before a new digit comes down, the remainder is smaller than the positive divisor—the failed trial saw to that. For divisor eight, the largest remainder is seven, and even bringing down nine makes only seventy-nine: less than ten eights. For any positive divisor, multiplying a smaller remainder by ten and adding at most nine still leaves less than ten copies of that divisor. Nine successful trials are the most a round can have; one further trial must fail. The requirement that the divisor be positive matters. Subtracting zero would keep our worker occupied without getting the pencil any nearer the answer.

Each trial crosses finitely many columns. Each copy crosses finitely many digits. A round contains only finitely many trials. Consequently every particular answer digit arrives after finite work, even when the entire decimal never ends. This is what we will need from a printer, rather more than the ability to keep somebody busy forever. The appendix's full division sheet makes a few economies: its cards can read the accepted remainder and divisor where they already lie, and it reuses one square for the current borrow. The fresh trial answer still gets its own paper, and a failed trial still leaves the accepted remainder alone.

In 1936, Turing compared the squared paper of his human computer to a child's arithmetic book. His analysis asked which symbols the person observes, which marks can change, and where attention moves. He then simplified the paper to a strip of squares.[2] We have kept a more familiar desk, but the little actions have emerged from the same activity. The address numbers themselves can be changed by ordinary



Alan Turing
(1912–1954)

[2]Turing, "On Computable Numbers," section 9, pp. 249–251.

addition and subtraction. A fixed collection of
place cards can serve us however far the work spreads.

Turing's table includes the current *state*, meaning the stage of the directions, alongside the symbol under attention. An entry specifies a new symbol, a move, and a next state:

state	symbol	write	move	next state
a	0	1	right	b
a	1	0	left	a

At stage `a`, a zero asks for one, a move right, and a jump to stage `b`. A one asks for zero, a move left, and a return to `a`. Our worker can follow any finite table like this: the state picks a part of the sheet, the observed symbol picks an entry, and its writes, moves and jump carry out the change. Following the addition tables has already supplied the method.

A finite sheet of instructions is a *program*. It has finitely many card names and uses a fixed finite collection of marks. Each jump names a line on the sheet; otherwise we continue to the following line. `Halt` tells the worker to stop. Skill and imagination can go into devising a program, but following it need not reproduce the thoughts of its inventor. That is why somebody else can take the chair.

Just before we copied three after 15., the desk contained the accepted six, the rejected scratch work and the place cards. Add the next instruction to this description and we have a *configuration*: enough to resume the calculation. After the write, three appears at $(0, 3, 0)$ and the next instruction changes. After the move, the three remains and `OUT` says $(0, 4, 0)$. A wrong digit or a moved remainder is now a specific disagreement with what the instruction allowed.

We can record configurations in sequence, making a *trace*. Each configuration needs only a finite list of written squares and their marks, the addresses on the cards, and the next instruction. A trace is therefore a sequence of finite sequences. To check one step, inspect the changes the instruction permits and check that the rest stayed put. This will let us examine calculations as well as perform them.

The worker can leave again. Our directions will find the next digit, and the next, without further advice. Yet what lies on the desk at any moment is still a finite beginning. We have learned how to keep writing a decimal. We have not yet asked how much an unfinished one can tell us.

Chapter 2

What the digits tell us

At 15.3, our division had six left over. Six looks an extravagant remainder for an answer already this close to 15.375. It belongs to the enlarged calculation on the scratch paper, where we have brought down a zero. Put the point aside for a moment and check the figures:

$$1230 = 8 \cdot 153 + 6.$$

Eight copies of 153 account for 1224; six makes up the rest of 1230. Eighty is eight times ten. Dividing the whole line by eighty removes the extra factor of ten from 1230 and leaves it divided by eight: 123/8. In the middle term, the eight cancels, leaving 153/10 = 15.3. The remaining six becomes six eightieths:

$$\frac{123}{8} = 15.3 + \frac{6}{80}.$$

That final term is 0.075. There was more information on the desk than the answer row alone disclosed.

Now bring down the next zero. The checking equation grows tenfold, just as the remainder grows from six to sixty:

$$12300 = 8 \cdot 1530 + 60.$$

Seven eights come out of sixty. Transfer them to the part already accounted for, leaving four:

$$12300 = 8 \cdot 1537 + 4.$$

This time the check, divided by eight hundred, says

$$\frac{123}{8} = 15.37 + \frac{4}{800}.$$

We could turn the four into 0.005, finish the division and go home. But even without doing that, we know the missing contribution

is less than 8/800, one hundredth. The answer lies between 15.37 and 15.38, short of the upper end.

The same account balances at every round. Let P be the integer we started with, Q the positive divisor, and R the remainder. After k digits beyond the point, remove the point from the answer so far and call the resulting integer B. Then

$$10^k P = QB + R, \qquad 0 \le R < Q.$$

Here 10^k is one followed by k zeros: for two decimal places, it is one hundred. The letters side by side mean multiplication. The last inequality records the unsuccessful trial: the remainder could not supply another copy of the divisor. Bringing down zero multiplies the account by ten; each successful subtraction moves a copy of Q out of the remainder and into the answer. Neither operation disturbs the equality.

Divide this account by $Q \cdot 10^k$, as we divided our example by eight hundred:

$$\frac{P}{Q} = \frac{B}{10^k} + \frac{R}{Q \cdot 10^k}, \qquad 0 \le \frac{R}{Q \cdot 10^k} < \frac{1}{10^k}.$$

The first term puts the point back into the written answer. The second is the amount still missing. It is smaller than one unit in the last written place—a tenth after one digit, a hundredth after two. This relation is the division's *invariant*: a fact kept true while the figures change. Later we will need the fact when those figures are far too large for anyone to inspect.

For now, cover the scratch work and look only at a beginning such as 0.37. If these are the first two digits of some endless decimal, its number cannot be below 0.37, nor above 0.38. Reveal one more digit, four, and it lies between 0.374 and 0.375. A digit has narrowed a whole interval:

digits written	lower end	upper end
0.3	0.3	0.4
0.37	0.37	0.38
0.374	0.374	0.375

We include both ends here. We will shortly meet the nines that reach the upper one. Each interval sits inside the preceding interval, and its width is a tenth of the preceding width. An entire decimal gives an interval at every length.

The real number line has no holes: these nested, closed intervals have a point in common.[1] They cannot keep two distinct points in common. However small the distance between two points, eventually one of the widths $0.1, 0.01, 0.001, \ldots$ is smaller. An interval that narrow cannot contain both. The one point left in all the intervals is the real number the decimal names.

Nothing in that description supplies a person with directions for finding the next digit. The decimal is a sequence of digits; the number is the point it names; a program for producing the sequence is something further. Long division has handed us all three together. Its invariant keeps the fraction inside every interval, so the digits name precisely the fraction we meant to calculate. We can watch that program closely enough to discover another feature of its answers.

Divide one by ninety-nine. A first zero brought down makes ten, too small to supply a ninety-nine. Write zero and keep ten. The next zero makes one hundred; one ninety-nine fits, leaving one. We are back to the old remainder:

old remainder	bring down zero	digit	new remainder
1	10	0	10
10	100	1	1

There is no occasion for a new answer. The divisor is still ninety-nine, the next zero is as good as the previous zero, and the remainder is one again. Our two lines repeat, giving $0.010101\ldots$. The worker could conceal the answer written so far and still know what to do.

For 123/999, a slightly longer trip comes home:

old remainder	bring down zero	digit	new remainder
123	1230	1	231
231	2310	2	312
312	3120	3	123

[1] This is the completeness property of the real line in its nested-interval form. When the widths also shrink to zero, the common point is unique.

The return of 123 as remainder starts the same three lines again. The desk itself has not returned to its old condition: three more digits are written, the answer card has moved, and there is more scratch work. But none of those changes helps choose the next digit. Once the original input has ended, only the remainder changes among the things that do. The answer continues $0.123123\ldots$.

Could some fraction keep finding new remainders forever? A divisor of Q allows only $0, 1, \ldots, Q-1$. After $Q+1$ remainders have been recorded, two entries must agree: there were only Q values to distribute among them. From that repeated entry onward the calculation retraces a route. Its digits repeat a fixed block forever. Zero is a possible returning remainder too; its repeated block consists of zeros.

The route may take a while to find its cycle. With $1/6$, the first zero makes ten; one six fits and four remains. The next zero makes forty; six sixes fit and four remains again. This four keeps returning, so the expansion is $0.1666\ldots$. The first one never returns. A fraction—a *rational number*—therefore has an *eventually* repeating decimal. The first few digits may be an approach to the repeating part.

We can recognize a fraction by the same sign of repetition, working backward. Give the name x to $0.123123\ldots$. Moving the decimal point three places right puts one full block before it:

$$\begin{aligned} 1000x &= 123.123123\ldots, \\ x &= 0.123123\ldots, \\ 999x &= 123. \end{aligned}$$

Subtracting cancels the equal tails. We have $999x = 123$, so $x = 123/999$. A block of any length gives the same maneuver: move the point past one copy, then subtract. The tails are equal numbers, and an ordinary subtraction removes them. Infinity has not required us to reach the far end with an eraser.

An unrepeated beginning does not spoil this. In $0.5123123\ldots$, first remove the initial five tenths. Multiply what remains by ten and the repeating $0.123123\ldots$ appears. Put the pieces back:

$$0.5123123\ldots = \frac{5}{10} + \frac{123}{9990}.$$

These two fractions can be put over a common denominator. Any finite beginning can be removed and restored in this way, so every eventually repeating decimal names a fraction.

Try the cancellation with a block consisting of a single nine:

$$x = 0.999999\ldots,$$
$$10x = 9.999999\ldots.$$

Subtract: $9x = 9$, and $x = 1$. The nines have reached one. This is an unwelcome result if we have been imagining an indefinitely long but still unfinished row, always waiting for its last tiny improvement. There is a gap after any finite row of nines. One nine leaves a tenth, two leave a hundredth, three leave a thousandth. The question is whether a positive gap can remain for the number named by the whole sequence.

Suppose one does. Keep writing nines until their remaining finite gap is smaller than that supposed final gap. The finite beginning has already passed the alleged final value, although all its later digits can only increase it. That is impossible. Every proposed positive gap fails, and the real number is one. The intervals say the same thing: their upper end is always one, and their lower ends leave room for no smaller common point.

One number has two names, $0.999\ldots$ and $1.000\ldots$. At a smaller scale the same meeting occurs:

$$0.249999\ldots = 0.250000\ldots.$$

A program can write 24 after the point and continue with nines; another can write 25 and continue with zeros. There is no reason to confiscate either instruction sheet. The marks are different, but the real number they name is the same.

To see whether there are other kinds of duplicate name, compare two different expansions at the first place they disagree. Suppose one has four in the hundredths place and the other five, after identical earlier digits. The smaller beginning is a hundredth behind. Its later digits can supply at most $0.00999\ldots = 0.01$, just enough to catch up if every one of them is nine and every later digit of the larger beginning is zero. Lower any of those nines, or

raise any of those zeros, and the gap remains. If the first differing digits were two apart, even all the nines could not close it.

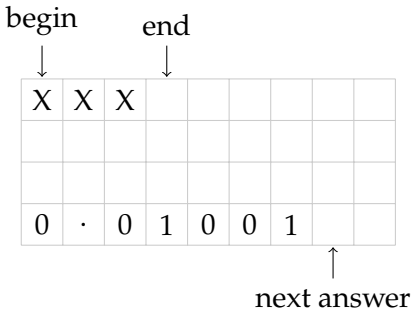
Nothing here depends on choosing hundredths. After any place, the largest possible tail is one unit of that place. Duplicate names must therefore part at neighboring digits, with all nines after the smaller and all zeros after the larger. A positive terminating decimal has exactly this second name: lower its last nonzero digit by one and follow it with nines. For $0.25000\ldots$, that gives $0.24999\ldots$. Zero has only its all-zero nonnegative name. Ignoring unnecessary zeros to the left of an integer part, there can never be more than two names.

Thus *every* decimal name of a rational number eventually repeats. Long division supplies one; the only possible alternative replaces a zero tail with a nine tail, which repeats as well. We now know what a decimal must do to get away from all fractions: it must defeat every possible repeating tail.

Our worker has more paper than a long-division remainder can use. Let us put that space to work. On a separate row write one tally, `X`. Cross the row, writing one answer zero for each tally, and then write an answer one. Add a tally before crossing again. The directions stay the same; the row they encounter gets longer:

counting row	new block	answer so far
`X`	`01`	0.01
`XX`	`001`	0.01001
`XXX`	`0001`	0.010010001

The answer begins with 0.. On the third crossing the paper looks like this, with the next answer square waiting below and the three tallies above:



The arrows show places saved on cards. `BEGIN` holds the first tally's place; `END` holds the blank just after the last. A working card, `SCAN`, will cross between them. `Q` holds the next answer place. These are jobs our cards already know how to do. The crossing itself takes six lines:

```
0 nextblock: Place(SCAN,BEGIN)
1 zeros: IfSame(SCAN,END) Goto one
2 Write(Q,0)
3 Move(Q,0,1,0)
4 Move(SCAN,0,1,0)
5 Goto zeros
```

Start `SCAN` at the first tally. It is not yet at the saved end, so a zero goes into the answer and both working cards move right. The test fails at the second tally and at the third, giving two more zeros. Only then do the places agree. The jump to `one` leaves the completed run of three zeros behind.

At `one`, write an answer one and move `Q` right. Put a new tally at `END`, move that end card right, and return to `nextblock`. The next crossing finds four tallies, and therefore writes four zeros before its one. The full program, including the opening 0., is in Appendix A. Each crossing finishes: the tally row at that moment is finite. Each adds a block, so every particular digit of the resulting decimal eventually arrives:

$$0.01001000100001000001\ldots.$$

Could a repeating pattern be hiding farther along, after the portion we have inspected? Assume there is one, beginning at

some point, and call its length m. If its block contains a one, every copy of the block contributes that one. Once repetition has begun, there cannot be m consecutive zeros: any m consecutive places contain a whole cycle's worth of digits. Our tally row will eventually make a run longer than that, beyond the proposed start of repetition. The block cannot contain a one.

Then it must consist entirely of zeros, since our printer uses no other digits. But every finite crossing ends with another one. There is no last one after which the zeros could settle in forever. This second possibility fails too. We have ruled out every repeated block, without waiting to see where it might appear. The decimal is irrational.

The proof needs both features: longer and longer zero runs, and ones that never cease. The number $0.10000\ldots$ has plenty of zeros but only one one; $0.010101\ldots$ has endlessly many ones but only short gaps. Our growing row keeps both requirements alive. It is a very small change to the desk for an escape from every pair of integers.

We have also begun doing something a printout cannot do for us. No finite inspection shows that the gaps will always grow. An argument about the instructions does: another tally is added after each finite crossing, and the next crossing writes one zero per tally. We can draw conclusions about places the worker has not yet reached by following what a return to those instructions must accomplish.

Call a sheet a decimal printer if it settles digits in order, never changes a settled digit, and eventually reaches every particular place. Long division qualifies, including its continued zeros after a terminating answer. So does our tally sheet. A sheet that produces a hundred digits and then spends forever trying to find the next one does not. Its hundred digits may be excellent; its decimal is still missing almost all of itself.

We call a real *computable* when some finite sheet prints a decimal name for it. This follows the digit-writing approach of Turing's original account. Choosing permanent digits imposes a particular obligation. An approximation close to 0.5 may leave open which side of 0.5 the true number occupies. A better approximation can

leave it open again. A printer asked to choose the first digit cannot put "nearly five" in the square.[2]

Both names of a terminating value remain legitimate outputs. We will mostly watch fractional expansions beginning with 0., including the all-nines name of one. What matters about a sheet is that it supplies a whole such name, rather than continuing indefinitely to reconsider what it might write.

How many sheets are possible? Begin with an alphabet consisting only of `a` and `b`. Its short texts can be arranged in levels:

length	texts
1	`a, b`
2	`aa, ab, ba, bb`
3	`aaa, aab, aba, abb, baa, bab, bba, bbb`

Each level finishes before the next begins. The alphabet needed for our directions is larger, but still finite. List every text of length one, then every text of length two, with a fixed alphabetical order within each length, and so on. Every finite program appears at a finite place in this numbered inventory. That is what it means for the collection to be *countable*.

Much of this inventory is nonsense. We can reject misspelled instructions and jumps to missing lines by inspecting a finite sheet. A correctly written program is harder to classify. It might halt, or go round forever without printing, or produce six digits and never reach the seventh. Its entry in the inventory does not tell us which future it has. We have counted the texts without acquiring a list of just the productive printers.

Still, each productive sheet names at most one real. Give each computable real the inventory number of the first sheet that prints a decimal name for it. Two different reals cannot claim the same sheet, so this assigns them distinct positive integers. Some positions are unused, and several sheets may print the same real; neither circumstance creates extra reals needing numbers. The computable

[2]Modern definitions use approximations with prescribed errors and describe the same individual computable reals. There is, however, no general algorithm converting a supplied approximation procedure into a decimal printer. See Hirst in the source notes.

reals are therefore countable. Knowing that these positions exist does not give the worker a test for identifying them in the inventory.

Our growing-gap decimal has a place among them. The tally row escaped the fractions by giving the old worker a new job; we would like to know how far all possible jobs can take that worker.

For the next experiment, the worker will have to share the pencil. Someone else will be allowed to insert digits between our blocks, spoiling the number we intended to write. The proof of irrationality may be harder to spoil.

Chapter 3

Drawers and games

You write two. I add seventeen. You add zero, zero, four. Between us we have made the beginning

$$0.2 \quad \longrightarrow \quad 0.217 \quad \longrightarrow \quad 0.217004.$$

Neither of us had to approve the other's digits. Now imagine that we want incompatible things from this collaboration. You want the whole decimal to name a fraction. I want it to be irrational. We can each write as many digits as we like on a turn, provided the block is finite and contains at least one digit. Then we have to hand the pencil back. The earlier writing stays.

The number on the table has acquired two authors and no editor. After 0.217004, zeros forever would make a fraction. A growing-gap tail would make it irrational. Both futures are available, but neither player can simply choose one: the other will interrupt. The question is what either of us can make happen despite those interruptions.

There is no last turn. Because each move adds digits, every particular position eventually fills, and the whole play names a real between zero and one. The all-nines play names one. Our rule gives every completed infinite play a winner, even though the completed page will never lie between our coffee cups.

Coffee has some claim to a place in this picture. In the Scottish Café in Lwów, mathematicians including Stefan Banach and Stanisław Mazur discussed problems recorded in a notebook now known as the *Scottish Book*. Mazur's Problem 43, from 1935, offered a bottle of wine for a question connecting a game of shrinking intervals with a mathematical notion of smallness. [1] Appending a block of digits is our way of choosing a smaller range of possibilities.

[1] Problem 43 in Mauldin's edition of the *Scottish Book*.

The player gets to decide how much smaller before giving up the pencil.

Take the irrational player's seat. On the first turn write a zero and a one; on the second write two zeros and a one; on the third, three zeros and a one. Continue lengthening the zero block. The other player can put a thousand sevens between our turns, or attempt to restore a pattern with an elaborate block of their own. They cannot remove our zero runs, whose lengths keep growing, and they cannot make our successive ones cease.

The proof from the previous chapter survives. A repeating block containing a one cannot allow arbitrarily long stretches of zeros. A repeating block containing no one cannot allow ones arbitrarily far along the page. Our moves ensure both, whatever else appears between them. We can force irrationality while leaving much of the actual number to somebody else. The opponent can spoil our intended decimal without spoiling the reason it escapes fractions.

Hold the page at 0.2. Imagine a drawer labeled [0.2], containing every complete decimal paper that could continue this beginning. It holds $0.20000\ldots$, $0.217004\ldots$, $0.29999\ldots$, and every other way of going on after the two. The label tells us what is fixed; inside are all the possibilities still unsettled.

Sort these papers by their next digit, and the drawer divides into ten smaller ones:

$$[0.20],\ [0.21],\ [0.22],\ [0.23],\ [0.24],$$
$$[0.25],\ [0.26],\ [0.27],\ [0.28],\ [0.29].$$

The drawer [0.21] opens into [0.210] through [0.219], each with ten more inside. A finite label never gets us to the back of this cabinet. Our sample play has selected three nested drawers:

$$[0.217004] \text{ inside } [0.217] \text{ inside } [0.2].$$

A move of one digit goes one level deeper. A move of a thousand digits goes a thousand levels deeper. However far the other player goes, their chosen drawer remains inside the one we last chose.

The objects in these drawers are papers, which matters at a shared edge. [0.20] contains $0.209999\ldots$, while [0.21] contains



Figure 3.1: The drawer [0.2] contains ten drawers, [0.20] through [0.29]. Opening [0.21] reveals its own ten drawers. Each label fixes a finite beginning and leaves all later digits available.

0.210000 . . .. The two papers name the same real. Their numerical ranges touch, although the papers themselves have different second digits and sit in different drawers. An event about the number 0.21 has to count both papers; an event about the next written digit concerns the particular paper we are making.

By an *event*, we mean something that can happen in the course of play, or a property of the complete decimal. "A seven appears" is an event. So is "sevens keep appearing, however far along we look." Our move seventeen settled the first: the seven is visible. It did not settle the second. We may have written the only seven the number will ever have. Events need not reveal themselves all at once, or at any finite point.

Choose an event before play. The first player wants it to happen; the second wants it to fail. It depends on the digits or their value,

rather than on which player wrote which mark. Our first event was rationality. We defeated it by spoiling every repeating tail. The drawers suggest a different attack, one that will work against lists much larger than the fractions.

Try to exclude just one half. Suppose the current page begins 0.49. Writing another nine keeps us away from $0.5000\ldots$, but it leaves the other name, $0.4999\ldots$, perfectly possible. Write zero instead. Now the whole drawer [0.490] misses one half: none of its papers starts with five, and none has the nine required at the third place of $0.4999\ldots$. Every later block stays inside this safe drawer. We have finished with one half, however many digits remain unwritten.

One third is even easier. If the paper says 0.3, write four. There is now a permanent disagreement with $0.333\ldots$. The other player may answer with a thousand threes, but the four in the second place does not move. A very long reply need not be an effective reply.

We can give every fraction such a turn. Make a list: denominator one with its possible numerators between zero and one; denominator two with numerators zero, one, two; denominator three with zero, one, two, three; and so on. Each stage is finite, and every fraction between zero and one occurs. Repetitions such as 1/2 and 2/4 do no harm. Beside each entry put all its decimal names beginning with 0.. Zero contributes its zeros, one its nines, and one half contributes both names. This is still a numbered list.

On our first turn, compare the shared page with the first name on the list. If they already disagree, add any digit—a move is required—and pass the pencil. If they still agree, calculate the next digit of that name and write a different digit. On our second turn do the same for the second name. Every particular calculation finishes: long division gives the needed finite beginning, and its returning remainders reveal the zero-tail cases with a second, nine-tail name.

By our millionth turn, the millionth name will have a disagreement on the page. That turn is distant, but only finitely many moves precede it, all finite. The earlier disagreements have survived. Every name eventually gets its own turn, and so none can be the name of our completed number. We have another winning

strategy for irrationality. This one relies on a scheduled escape from individual targets, rather than a special feature of repetition.

René Baire's idea lets us escape targets containing many possibilities at once. Baire studied where continuity can fail: exceptional sets whose arrangement can matter more than their size as a count of points. His work in the late 1890s developed a way to describe smallness through the room remaining around such sets.[2]



René Baire (1874–1932)

Give the first player the event "no seven ever appears." From any beginning, the second player can append seven. Every paper in the new drawer contains that seven, so none satisfies the event. If a seven was there already, any further move preserves the defeat. The event allows vastly many decimals, but it leaves us this way out from every drawer.

An event is *nowhere dense* when, from any finite beginning, we can append a finite block after which *every* continuation avoids it. A whole smaller drawer is clear. This is stronger than finding one possible safe paper. With only one safe paper, we might hand the pencil to our opponent and watch them choose a different continuation. With a safe drawer, all their choices stay safe. Our escape lasts beyond our turn.

Equality to one specified real is such an event. If the real begins 0.2531415 . . . and the shared page says 0.25, writing seven excludes it. If the page already says 0.31, it is excluded before we start. In general a real has at most two decimal names, so at most two of the ten immediate subdrawers can contain its papers. There is always another subdrawer to choose.

[2]Baire's 1900 paper, "Nouvelle démonstration d'un théorème sur les fonctions discontinues," returns to questions about continuity studied in his 1899 thesis. The decimal game is a later presentation of the category method.

That last argument did not require us to know the real's digits. It proves that some safe labels exist, even when we cannot work out which they are. For one half, finding a safe label was an ordinary finite calculation. For an arbitrary real, it may not be. Keep the difference in view: room to move and instructions for finding the move are separate gifts.

Now change the event to "only finitely many sevens appear." One seven will not defeat it. A million sevens will not defeat it either; we might have supplied a particularly tedious finite collection of sevens, followed by zeros forever. There is no finite block that settles the matter in our favor. Yet writing one seven on every turn plainly wins against this event. Baire's next step describes exactly how such a sustained escape works.

Break the event into these possibilities:

F_1: there is no seven after position one.

F_2: there is no seven after position two.

F_3: there is no seven after position three.

Continue with every positive integer.

If the whole decimal has only finitely many sevens, choose a position beyond its last seven; the corresponding F_n holds. If it has no sevens, all the F_n hold. Conversely, if any F_n holds, every seven is confined to its first n positions, so there can be only finitely many. The original event happens exactly when at least one entry on this list happens.

Each entry is nowhere dense. To defeat F_n, continue far enough to pass position n and write seven. Whatever follows, a seven now lies beyond the alleged boundary. We can defeat F_1 on one turn, F_2 on another, and continue. One turn cannot abolish the possibility of a last seven. The sequence of turns can prevent any seven from being the last.

A countable union of nowhere-dense events is called *meager*. "Countable" gives the pieces numbered places; "union" means that at least one of them happens. Finite lists are allowed, and pieces may overlap. Our event of finitely many sevens is meager, though it remains possible after every finite beginning: fill the

remaining places with zeros. Meagerness does not say that an event is impossible, or attach a numerical probability to it. It says its ways of happening can be put into a countable list of pieces that individually leave room for escape.

For the proof, put the opponent's pencil aside for a moment. Take any meager event, with pieces $F_1, F_2, F_3, \ldots$, and any starting drawer, perhaps [0.21]. Because F_1 is nowhere dense, append a nonempty finite block entering a drawer clear of F_1. From there choose a further drawer clear of F_2. It is still inside the first safe drawer, so the escape from F_1 has not been lost. Continue with F_3, and then the next piece.

With our seven events, successive choices could look like this:

stage	beginning after the choice	targets secured so far
1	0.217	F_1
2	0.217047	F_1, F_2
3	0.217047387	F_1, F_2, F_3

The last seven in each line lies beyond the position named by that stage's target. In fact the first seven, at position three, already defeats F_2 as well as F_1. The schedule need not refuse such good fortune; it merely ensures no entry is forgotten. For other events the chosen blocks will be different. What matters is that each new line extends the whole previous line and secures its assigned escape.

Each stage adds at least one digit, so these beginnings fit together into one endless paper. Choose any entry F_n from the list. Stage n put the paper in a drawer entirely clear of that entry, and every later stage stayed there. Hence the whole paper avoids F_n. Since this works for any entry we choose, the paper avoids all the pieces and their union. We do not need a last stage that certifies infinitely many escapes. Each possible accusation already has its own finite stage at which it was ruled out.

This is the *Baire category theorem* in the form our decimals need:

> No meager event fills a drawer. After any finite beginning, there is a complete continuation outside it.

For events concerning real values, each target includes all allowed names of the value. An all-nines continuation is still one of the

papers we have kept inside our safe drawers. Shared endpoints do not open a hidden way back into an excluded event.

Now give the opponent the pencil again. After our escape from F_1, they append 999999. Our page, formerly 0.217, becomes 0.217999999. Does the next escape still exist? Yes: F_2 is nowhere dense in *every* drawer, including the drawer this new block has chosen. It supplies a further safe subdrawer. All earlier escapes remain intact. The opponent can make the next move start somewhere else, but cannot make it start somewhere with no way out.

This gives a winning *strategy* for the second player whenever the first player's event is meager. A strategy supplies a legal finite move for every history that could lead to the player's turn, including how the earlier digits were divided into moves. At our nth turn, escape F_n from the current drawer. The proof applies to every legal opposing play, not merely the particular reply 999999.

If several blocks work, choose the first in a fixed ordering: shortest length first, then alphabetical order among equal lengths. The nonempty collection of successful blocks has a first member, so this specifies one reply. But a worker cannot necessarily recognize which blocks succeed. A mathematical description of a strategy need not be a program for carrying it out. We have retained the distinction that arose when we excluded an arbitrary real.

The theorem already tells us that the reals cannot all fit on a numbered list. Suppose someone hands us such a list. Let F_1 say that the completed number equals the first entry, F_2 that it equals the second, and so on. Equality to a specified real is nowhere dense, so the event "our number occurs on the list" is meager. Baire's construction escapes it. The starting drawer could have any finite label: even after a thousand agreed digits, a real missing from the list remains available.

This is *uncountability*. The list may contain fractions, irrationals, familiar constants, or numbers whose digits nobody can calculate. Every entry still occupies at most two papers, and every turn still leaves a whole drawer free of its target. The second player can force a result outside the list while an opponent supplies all the intervening blocks. A list has given each of its entries a number; we use those numbers to arrange its defeat.

The first player's opening move can confer an advantage too. Suppose there is a drawer in which the ways for their desired event to *fail* form a meager collection. They enter that drawer immediately. On each later turn they escape another nowhere-dense piece of the failure event. The opponent cannot leave the starting drawer or undo earlier escapes. The finished play avoids every listed way of failing and therefore satisfies the first player's event.

Such an event is called *comeager in that drawer*: its complement there, the event of failure, is meager. For "infinitely many sevens appear," the failure event is exactly the one we have just analyzed, "only finitely many sevens appear." Thus infinitely many sevens is comeager in every drawer. Either player, if given the job of ensuring it, could write a seven on every turn. No particular seven guarantees victory; the rule guarantees that no seven is the last one we contribute.

There are two other descriptions worth attaching to these familiar possibilities. The event "a seven appears" is *open*: whenever it happens, a finite beginning eventually shows enough to guarantee it. The first seven is the witness. "Infinitely many sevens appear" is not open. No finite collection of sevens guarantees another one.

An event is *dense* when it remains possible in every drawer. Infinitely many sevens is dense: we can continue with sevens. Finitely many sevens is dense too: we can continue with zeros. These incompatible events both remain possible after every finite move, although a player can force the first by returning with another seven. Possibility alone says little about who controls the outcome.

When an event is both open and dense, something stronger is available on a single turn. Density supplies a successful continuation in the current drawer. Openness says some finite part of that continuation already guarantees success. Append that part, and the entire new drawer satisfies the event. We will meet this kind of opportunity again near the end of the book: a requirement may be reachable from wherever the work has got to, and a finite extension may secure it.

For our finite-block game, the winning conditions can be stated exactly. The second player has a winning strategy precisely when the first player's event is meager. The first player has one precisely

when their event is comeager in some drawer. We have constructed the strategies in these two cases. Chapter 4 of the *Technical Companion* proves the reverse directions. Writing a finite *block* on a turn is essential: the safe drawer may lie several digits away, and the player needs to reach it before the opponent takes over.

In the Scottish Book, Banach recorded an affirmative answer to Mazur's question, dated August 4, 1935. This family of games is now called the *Banach–Mazur game*. The link to Baire's smallness is the one we have followed: one player keeps making finite escapes, and the completed play avoids the countable collection. The wine was offered for a question whose answer concerns every possible reply, including replies long enough for the coffee to go cold.

Must one of the players always have a winning strategy? A game with such a strategy for one side is called *determined*. Every completed play has a winner by our rules; that does not yet put a way of ensuring victory into either player's hands. We will leave that further question here. There is a difficulty closer to our worker's desk, even in a game whose winning strategy we already know exists.

Let the first player now want the event "the resulting real is computable." There are countably many finite sheets, and therefore countably many reals their productive printers name. Include every allowed decimal name of each such real. Equality to each one is nowhere dense, so computability is a meager event. The second player has a winning strategy: they can force the decimal to escape every finite printer, including our growing-gap sheet and every long-division sheet.

Try writing that strategy down for the human computer. For fractions, we could calculate a target's next digit and disagree. The next program in our inventory writes six digits and falls silent. Its worker has not stopped moving. Perhaps the seventh digit will arrive tomorrow; perhaps the instructions send the worker searching forever. Our turn must end, and waiting forever is not a legal move. A fixed deadline does not solve it either: the digit might appear one step after the deadline. The inventory contains all program texts, not just the sheets that keep their promise to print a decimal.

Perhaps a cleverer strategy would avoid the waiting. Suppose there were any finite program carrying out a winning strategy for the second player. Give it an opponent with the simplest possible habit: write one zero on every turn. A single human computer can now play both sides. Write zero. Follow the strategy until its finite reply is finished. Write another zero. Follow the strategy again. The strategy must produce a legal reply on each turn; otherwise it was not the program we supposed it to be.

These combined directions form a finite instruction sheet. Every turn ends, every turn adds digits, and none of the digits is erased. So the sheet prints a complete decimal. Its real is computable. But this is a play the supposed winning strategy guaranteed would be noncomputable. The contradiction rules out the program. An opponent contributing nothing but zeros has exposed the limit of every proposed mechanical strategy, an unusually economical performance.

The winning strategy still exists. We proved there was a safe reply from every relevant drawer and a mathematical way to select one. What cannot exist is a finite sheet that finds those replies in every case. Our first escape from fractions could be performed by the worker; this escape from all printers includes the worker's own possible instructions among the things to be defeated.

After any finite beginning, both computability and noncomputability remain possible events. Zeros give a computable continuation. Baire's argument gives a noncomputable one. However impressive a finite page becomes, it has not decided which kind of future it has.

Six digits are still sitting beside the silent program. We could wait longer, but the additional silence would only enlarge the same uncertainty. Another possibility is to read the instructions themselves. Our worker has been given numbers as input; next we will give the worker a program, written out on the paper, and ask what it will do.

Chapter 4

Instructions about instructions

The seventh digit is taking its time. Six digits have appeared, the worker is still busy, and the supply of scratch paper is diminishing at a reassuring rate. We can see work being done. We cannot see whether it is getting anywhere. For long division we had a reason to wait: after at most nine successful subtractions, the next digit would be ready. An unfamiliar instruction sheet owes us no such courtesy.

This is awkward for our decimal game. To disagree with a digit, we need it to arrive. Waiting another hour does not distinguish a slow calculation from one that will spend eternity making preparations. Perhaps we could read the sheet instead. A jump back to the same jump is easy to diagnose. A more elaborate sheet might take a more elaborate diagnosis. Could we give a second worker directions for settling every case?

Let us ask for something definite. Our inspector receives a valid program, started without incoming data, and must eventually answer one of two questions: does it stop, or does it continue forever? The inspector must finish its inspection even when the inspected program does not finish its work. Merely running the program and watching it will settle the first possibility. It leaves us rather overqualified for the second: we already know how to wait.

There is a tempting way to embarrass such an inspector. When it says "stops," start an endless loop; when it says "continues forever," stop. Those responses are easy to write. The difficulty is getting the verdict to concern the very program that will respond. If the inspector predicts what one sheet does and another sheet does the opposite, nothing has gone wrong. We need to bring the description of the responding sheet within its own reach.

At present a sheet governs the worker's actions. It can also become an object on the desk. Write `Halt` on a scrap and ask

someone to copy it ten times. There is no reason for the copying to stop after the first word. The directions being obeyed say to copy; the direction being copied is just ink. This distinction is available to our worker too, provided we write the copying directions carefully enough.

Here is a small sheet to put under inspection:

```
0: Write(A,7)
1: Move(A,0,1,0)
2: Halt
```

Following it puts a 7 in the square on card `A`, moves the card one square right, and stops. Copying it produces three lines, including the line that says to stop. A copier saves the place just beyond the supplied text and works toward it. Spaces, punctuation, and line breaks are copied along with the other characters. An empty square inside the text cannot be mistaken for the end of the job.

We can go further than copying. Suppose we keep an account of what would happen if the supplied sheet were followed. At the beginning, card `A` points to its starting square and the paper is blank. After the first instruction, the account says that this square contains 7. After the second, it says that the card points one square farther right. The entry for the 7 stays where it was:

Next instruction	Position on A	Written marks
`Write(A,7)`	starting square	none
`Move(A,0,1,0)`	starting square	7 in that square
`Halt`	one square to the right	7 in the starting square

Now replace the last line with a small test:

```
2: If(A,7) Goto 4
3: Write(A,8)
4: Halt
```

At line 2, the card points to a blank square. The test fails, line 3 writes 8, and the finished row reads 78. Looking at the square the card used to indicate would give the wrong answer. Keeping track

of the place is part of following the instruction, just as it was when we carried a 1 into the next column of an addition.

The account itself may sit anywhere. We can write "7 at (0, 0, 0)" halfway down page twelve without transporting the original 7 to page twelve. A move changes the recorded address on a card; a write changes the recorded mark at an address. The record contains the configuration of the supplied computation, and each update must preserve everything the instruction leaves alone.

This is a *simulation*. One fixed worker can perform it for different supplied programs by looking up the next instruction and making the appropriate update. At any finite stage the record is finite, so looking through it takes finite work. Chapter 1 of the *Technical Companion* gives the scans and updates. We call the resulting procedure `Run`. If the supplied program stops, `Run` returns its result; if it continues, `Run` continues following it. Simulation gives us a way to use a sheet as input. It has not yet given us a way to predict its future.

To get the program's own sheet into the calculation, we will first make sheets for somebody else. The word `MATH` is a manageable customer. A printer for it writes M, moves right, writes A, moves right, writes T, moves right, writes H, and stops:

```
Write(OUT,M)
Move(OUT,0,1,0)
Write(OUT,A)
Move(OUT,0,1,0)
Write(OUT,T)
Move(OUT,0,1,0)
Write(OUT,H)
Halt
```

Here `OUT` begins at the result row. The letters come from the instructions, so this printer returns `MATH` whatever incoming word it is given.

Now put `MATH` on an input row and ask a different worker to prepare those instructions. The worker reads M and writes the command for writing M, followed by the command for moving right. The next letter supplies the next writing command. At

the end it finishes the sheet with a halt. Given `CAT`, the same preparation would produce a printer for CAT. Nothing in this job requires running the printer it has made.

We shall call the preparation `Print`. The name needs a little care: `Print(MATH)` is a sheet of instructions. *Running* that sheet returns the four letters. The distinction becomes useful when the incoming text is itself a whole program. `Print` then builds a printer for all its characters, including its commands, spaces, and final halt. When the printer runs, those characters are returned as text. Only the printer's own instructions are obeyed.

There is one other preparation to make. Put the MATH printer beside a program that copies its input and appends an exclamation mark. We want a single sheet that runs the printer first and passes the returned word to the exclamation program. Running that combined sheet gives

`MATH` $\longrightarrow$ `MATH!.`

The preparation, called `Combine(p,q)`, takes two program texts, p and q. It copies them into fixed surrounding directions: run the first; give its result to the second; return the second result. The copying finishes even if one of the supplied jobs never will. In that case the finished sheet is a finite set of directions for a very long engagement.

For example, let the first job simply halt, and the second jump back to itself. We can assemble them, set down our pencil, and go away. Someone who later follows the assembled sheet will reach the second job and stay there. Preparing the work and doing the work are separate events. Chapter 1 of the *Technical Companion* assembles this example down to its punctuation.

The punctuation matters because we will need the *same sheet*, not merely another sheet that behaves similarly. Fix one convention for every space, copied character, and instruction label. Under that convention, repeating `Print` or `Combine` with the same inputs produces exactly the same text. Their elementary expansions are fixed as well. We have three distinct jobs now: `Print` makes a printer; `Combine` joins two jobs; `Run` does the work described by a supplied sheet.

Alonzo Church studied calculation by manipulating expressions. His student Stephen Kleene found that such manipulations could bring a program's own description into its work. Our inspector has made this sound like an exotic requirement. The materials are the copying and assembly we have just performed.

First aim for something apparently simpler: a program that returns its own complete text. Writing "copy this program" begs the question. We have no instruction that points to *this*. Including a literal copy of the whole sheet seems to demand a copy containing the copy, which contains the copy. We can avoid starting that pile by making one ordinary preparation and using it twice.

Take a white sheet. It receives a program text and prepares a package with two parts. The first part prints the supplied text. The second part is that supplied text, to be run as a program on what the first part returns. In the notation we have earned, white performs

$$D(n) = \texttt{Combine}(\texttt{Print}(n), n),$$

where n is the incoming program text. White returns the package. It does not run it.

Try white on our exclamation program. The package white makes would first print the exclamation program's instructions, then give those instructions to the exclamation program. Its result would be the instruction text with an extra exclamation mark. The program has received its text from the printer in front of it. White did nothing more mysterious than make that printer and put the two jobs together.

White's own directions are a finite text too. Finish writing them, then supply that complete text to white. Save the package it makes on a blue sheet. Blue therefore contains these two jobs:

First supplied text	Second supplied text
`Print(white text)`	The white text itself

We have already completed one calculation: white, given white, made blue. Now run blue and put its result on a green sheet. Its first job returns white's text. Its second job runs white with exactly that text as input. But white given white is the calculation we just completed. It prepares blue.

Green consequently receives every character of blue. The text being returned is the text of the running program. Blue includes both jobs, all their surrounding directions, and the literal material they use; the reconstruction returns all of it. Here is the route again, with the colours kept in place:

$$\begin{aligned} \text{white, given white} &\longrightarrow \text{blue,} \\ \text{blue's first job} &\longrightarrow \text{white,} \\ \text{blue's second job, given white} &\longrightarrow \text{blue.} \end{aligned}$$

Nothing tells the worker to run the newly returned blue sheet. It has returned a result and can stop. Self-reproduction need not be an incurable condition.

Such a program is called a *quine*. If b denotes the white text, the blue text is $D(b)$. Running blue runs white on b, and white returns $D(b)$. The notation lets us check that the two occurrences of blue really are identical: the same preparation has received the same input. No instruction had to recognize itself.

Now we can put the recovered description to use. Suppose we want the running program to count its own characters. Change white so that, after preparing the package, it counts the characters in that package and returns the count. To make a *new* blue sheet, use the original package-making operation D: prepare a printer for the changed white text, then combine that printer with the changed white program. This assembly copies the text without executing it. Running changed white at this point would return a count, which would make a poor instruction sheet.

Now run new blue. Its first job supplies the changed white text; its second runs changed white on that text. White reconstructs precisely the new blue package and counts its characters. The instructions for doing the counting are included in what gets counted. We do not need to guess the final length and then revise the guess because writing it down has made the program longer.

Counting is just one possible use of the recovered text. Any mechanical task that accepts a program description can be put in that place. This is the form of Kleene's *recursion theorem* we



need: we can construct a program that performs the chosen task on its own complete description.

Let the chosen task be T. White still prepares a printer for its input and combines that printer with the input program. Instead of returning the package, it gives the package to T:

```
Call Print(INPUT, printer)
Call Combine(printer, INPUT, package)
Call Task(package, RESULT)
Halt
```

The named calls stand for the corresponding paper procedures. A call returns to the following line if its job finishes. The first two prepare text; only the third carries out the chosen task on that text. If the task never finishes, white never reaches its halt.

Call this finished white text b_T. Make blue in the original way, as $D(b_T)$, and call that blue text r. On running blue, its printer supplies b_T. White then prepares $D(b_T)$, which is r, and gives it to T. Thus the complete running program performs T on r, its own description. All the preparation ends before T begins. It adds work and scratch paper but no permanent decimal digits; afterward the resulting program has the same returned result or permanent output as T on that description.

We can finally submit the right sheet to our inspector. Choose T to request an inspection of its incoming program text and then respond contrariwise. Write Y for a predicted halt and N for a predicted endless run:

```
Call Inspect(INPUT, answer)
If(answer,Y) Goto Forever
Halt
Forever: Goto Forever
```

Kleene gives us a complete program that performs this task on its own text. The inspector's promise includes this program: it is a valid finite sheet, starting without incoming data, just like the other sheets it agreed to inspect.

Suppose the answer is Y. The program follows the jump and loops forever, making the answer wrong. Suppose the answer is N. It reaches its halt, again making the answer wrong. The inspector has promised to finish, so withholding an answer is not a third way to keep its promise. Both permitted answers fail for the very program they describe.

The contrary responses did not require ingenuity. The work was in arranging that the verdict reached its own subject. If a universal inspector existed, we could copy its directions into this construction and obtain a case it could not correctly settle. Hence it cannot exist. This is the *halting problem*: no mechanical method always finishes and correctly decides whether any supplied program will stop.

We have lost a universal method, not our ability to reason about particular sheets. A write followed by a halt stops; a jump back to itself does not. Nor does allowing incoming data repair the inspector. To ask about a particular program with a particular input, prepare a printer for that input and follow it with the program. The combined sheet contains the data and starts without any.

One direction of inspection still works: simulate the program and announce its halt if it reaches one. Every actual halt will eventually be announced. No corresponding moment arrives at which an endless computation has finished demonstrating that it is endless.

Return to the seventh digit. Given any program, make a new sheet that simulates it on private paper and writes a 7 only when that simulation halts. Someone who could always decide whether the 7 will appear could always decide whether the original program halts. Our missing-digit inspector is therefore impossible too. The unsettled work on the scratch pages has a direct effect on what we can know about the answer row.

We can hide it more effectively by keeping that row busy. Begin a decimal with 0.. Before each simulated working step, append a zero. If the simulation reaches its halt, append a single 1 and then zeros forever. Here a working step is one taken before reaching the halt instruction. A simulation with two such steps produces

$$0.0010000\ldots;$$

one with five produces

$$0.0000010000\ldots.$$

If it never halts, the row is all zeros. The simulation's own marks remain on its private pages; none is passed off as part of this decimal. Each simulated step involves a finite record and finishes, so in every case this new printer supplies every requested digit after finite work.

Set it beside the printer that writes zeros forever. The two numbers are equal exactly when the simulated program never halts. If it halts, the solitary 1 makes the first number positive, however far out that 1 appears. A method that always decided whether these productive printers name the same real number would therefore decide the halting problem.

Watch them for a while. Zero agrees with zero, then another zero agrees with another zero. Nothing is delayed. Each successful comparison is real evidence about that place and remarkably little help with the places still to come. The possible 1 can wait beyond every place we have checked. We have repaired the interruption in the printing without repairing the difficulty of knowing what the whole decimal is.

We could respond to a halt by starting the growing-gap decimal instead. Then the number would be zero if the simulation ran forever, and irrational if it stopped. Chapter 3 of the *Technical Companion* follows that variation and the related problem of recognizing endless decimal printers. For the equality question, a single possible 1 has already done all the damage required.

There is also a decimal in which each place asks its own halting question. List all valid program texts together with all finite inputs, using the length-and-alphabetical ordering of finite writing. The grammar can be checked without settling whether a program will halt. Put a 1 in place n if the nth program-and-input pair halts, and a 0 there if it does not. This defines

$$H = 0.h_1h_2h_3\ldots.$$

We have listed the questions and specified what their answers mean. We have not found a procedure for obtaining the answers.

Suppose nevertheless that a program prints H. For the twentieth halting question, wait for its twentieth digit. For the millionth, wait for its millionth. Every one of these particular waits must finish, because this is supposed to be a printer for the whole decimal. Reading the appropriate digit would then supply the universal inspection we have proved impossible. Thus no program prints H.

Could the same real have a second decimal name that is easier to print? Our ambiguity between a tail of zeros and a tail of nines does not help here. There are no nines in H, and there are infinitely many ones: even one immediately halting program, paired with all the different finite inputs, supplies infinitely many halting pairs. Its expansion is neither eventually zero nor eventually nine. This real has only the one decimal name, and that name cannot be printed by a finite program.

Baire gave us room outside a proposed list. Here is a particular occupant, defined using the very programs from which we hoped to make our list. Every digit has a precise mathematical meaning. What we cannot have is one finite sheet that keeps writing those meanings down as zeros and ones.

For an individual question, a proof might still succeed where watching a calculation fails. We can show that the jump back to itself runs forever by reading its one instruction. We do not have to watch the eternity it describes. A worker could search through proofs, looking for an answer about its own work. Proofs are finite writing; our programs have learned to read finite writing. We should now ask what happens when the conclusion becomes an instruction's input.

Chapter 5

Proof on paper

Watching the paper cannot always tell us whether another digit will come. A proof could tell us. The familiar jump back to itself can be dismissed in a sentence: it returns to the same instruction, so it will never reach anything else. We know more from that sentence than from watching a thousand jumps. For a less obliging program, the argument might occupy a hundred pages. If those pages finally settle the question, we need not wait for the calculation to settle it.

Our worker, however, must be able to recognize what the pages establish. Put a long division beside an explanation of its answer. In the calculation, a misplaced carry has a location. In the explanation, an omitted step may be hiding in "clearly." A reader can sometimes repair it. Our worker has no instructions for being that kind of reader. Whatever licenses the next line must be available on the paper, in a form the worker can check.

David Hilbert put this demand at the centre of his study of mathematics' foundations. A formal proof is a finite piece of writing governed by exact rules. Finding it can take imagination; checking it need not repeat the discovery. The person checking could be unimpressed by the theorem, misunderstand its importance, and still perform every check correctly. Insight has done its work and left something behind.

This gives our programs a new object to read. It also gives them something to react to. Imagine directions that say: look for a proof that this program cannot return 314; if one appears, write 314 and stop. Kleene's construction can arrange that "this program" means the complete sheet doing the searching and writing. The proof would trigger the event it excludes. Before deciding whether that can happen, we must give the worker something more precise to recognize than an emphatic use of the word "cannot."

The three instructions from the last chapter supply a small completed computation to work with. Write 7, move right, halt. Its trace records the configuration at each instruction boundary:

Entry	Next instruction	Card A	Marked squares
0	`Write(A,7)`	$(0, 0, 0)$	none
1	`Move(A,0,1,0)`	$(0, 0, 0)$	7 at $(0, 0, 0)$
2	`Halt`	$(0, 1, 0)$	7 at $(0, 0, 0)$

Call the record r. A checker can start at its first entry, compare each pair of neighbouring entries, and see that the last one reaches the halt. The move must leave the 7 at its old address. A record that carries the 7 along with the card has recorded a different action, however plausible its final answer looks.

For this program, "it halts" has a witness on the desk: r. For the program searching for a proof, the same kind of claim asks for a record of unknown length. We need to say that *there is* a record, that its beginning is right *and* its successive changes are right, and perhaps that there is *not* any such record. Our proof language will grow from these needs. It must let us describe a completed calculation without first having completed it.

The records themselves have many pieces: marks, addresses, place cards, configurations, and the positions of those configurations in a trace. We will collect pieces in sets. At first, picture a bag with objects as members, some of which may be other bags. There is no left-hand end, and putting an object in twice does not make two copies of it. This will shortly be an inconvenience. For now, the useful fact is that we can put several things together and talk about what is in the resulting collection.

Write $a \in b$ when the object named a is a member of the object named b. If we know $a \in b$ and also $b \in c$, we can write both facts on one line:

$$\frac{a \in b \qquad b \in c}{(a \in b) \wedge (b \in c)}.$$

The sign $\wedge$ means "and." Above the bar are the two facts available; below it is the statement we may infer. Such an arrangement is a *proof tree*. Its branches record the reasons for what appears beneath

them. We can also take either part of an "and" statement back out and use it by itself.

Notice what this tree has not done. It has not concluded $a \in c$. Something inside an inner bag need not be a direct member of the outer bag. The rule only preserves the two statements we supplied. A checker can verify that preservation by comparing the writing, without deciding what any of the bags contain. With A and B standing for whole statements, its general form is

$$\frac{A \qquad B}{A \wedge B}.$$

The statements could be about membership, or about the first two entries in r. A longer statement makes more writing to compare but does not change the rule.

We need rules for the experiments we make inside an argument too. Suppose e is empty and $e \in b$. Could b itself be e? Try that equality. Substituting e for b in our membership statement gives $e \in e$. But nothing belongs to e. The attempted equality has brought us to a statement and its negation, so we reject it.

The sign $\neg$ means "not," and $\perp$ records a contradiction. We can leave the failed trial visible while marking where it begins and ends:

Available: $e \in b$ and $\neg(e \in e)$.
$[b = e]_4$ temporary assumption
  $e \in e$ replace equals in $e \in b$
  $\perp$ contradiction
$\neg(b = e)$ close assumption 4.

The label 4 is there so the checker can identify the assumption being closed. We say it has been *discharged*. The facts $e \in b$ and $\neg(e \in e)$ are still needed. The trial equality is no longer a condition on our conclusion.

Nor can we keep using $e \in e$, although it remains plainly visible on the page. That line depended on the equality we tried. If failed arguments were allowed to leave their consequences available for later use, mathematics would accumulate results at an impressive rate. The checker must therefore keep track of the assumptions

behind each line, removing only the ones a rule permits it to discharge.

We also allow the classical version of contradiction: assume $\neg A$, derive a contradiction, and conclude A. This is *reductio*, or proof by contradiction. Equality gives us two further familiar permissions: an object equals itself, and equals may replace one another in a statement. We have just used the second permission to expose the failed equality.

An existence claim changes what we may write in a different way. From $a \in b$, we may say that b has a member:

$$\frac{a \in b}{\exists x(x \in b)}.$$

Read $\exists x$ as "there exists an x." The object a is our example. The letter x shows the place in the condition that the example fills; it is not a newly discovered object named x.

Coming back in the other direction requires care. Suppose someone only tells us that b has a member. We may give an unknown member a temporary name, u, and reason from $u \in b$. We cannot choose a familiar object already on the desk and decide that it must be the promised member. If we knew that particular object was outside b, our choice would manufacture a contradiction where there was none. The bag could have had a different member all along.

The temporary name must be fresh, and anything carried out of its branch must be independent of which member it names. This condition sounds abstract until we try a whole proof. Assume that b has a member and e has none. Call these starting claims S and N:

$$S:\ \exists x(x \in b), \qquad N:\ \forall z \neg(z \in e).$$

The sign $\forall z$ means "for every z"; here it says that every object fails to belong to e. Name a member of b temporarily, then try making b and e equal:

Line	What is written	Why it may be written
1	$\exists x(x \in b)$	Starting assumption S
2	$\forall z \neg(z \in e)$	Starting assumption N
3	$[u \in b]_7$	Name a member promised by line 1
4	$[b = e]_8$	Try equality
5	$u \in e$	Replace b by e in line 3
6	$\neg(u \in e)$	Use line 2 for u
7	$\bot$	Lines 5 and 6 contradict
8	$\neg(b = e)$	Close assumption 8
9	$\neg(b = e)$	Close witness assumption 7

At line 8 the attempted equality has failed. We are still inside the branch where u names the unknown member. At line 9 we can leave that branch because the conclusion does not mention u. Whichever member b has, its existence prevents b from being the empty object e.

The last two lines say the same thing, but do not do the same work. Line 5 depends on assumptions 7 and 8; line 6 depends on N. Their contradiction therefore uses $N, 7, 8$. Closing 8 leaves $N, 7$. Closing the witness branch replaces 7 with the existence claim S that allowed us to open it. We finish with exactly S, N, our original assumptions. No particular member has been selected outside the argument.

Try carrying $u \in b$ out instead. The checker refuses: u is still in the proposed conclusion. The name was borrowed for the branch, and the conclusion is trying to keep it. In general the fresh name must be absent from the original existence statement, the conclusion carried out, and the other assumptions left open. All three conditions can be checked by looking through finite writing. A proof beginning with the claim "there is a completed record ending in 314" could likewise name the promised record temporarily and investigate its contents. That name would let the argument proceed. It would not perform the calculation or put the three digits on our worker's answer row.

Even replacing a name can change a statement if we ignore its surroundings. In $\exists x(x \in b)$, the x's are bound by "there exists";

b is free to name the bag. Suppose we want to use x as another name for that bag. Blindly replacing b produces $\exists x(x \in x)$, a claim that something belongs to itself. The name has been caught by the wrong job. First rename the bound letter, giving $\exists z(z \in b)$. Then the replacement gives $\exists z(z \in x)$, still the claim that our bag has a member. This is called avoiding *capture*. Renaming and checking where a letter is bound are more tasks for the worker's pencil, not appeals to insight.

Our decimal arguments already used two other familiar forms of reasoning. We divided the proposed repeating blocks into cases: those containing a 1 and those containing none. We also took an arbitrary denominator and examined its possible remainders. Both forms have a place among these rules.

For cases, suppose b contains a or contains c. In the first branch, a witnesses that b has a member; in the second, c witnesses it. Either way we reach the same existence statement. We need not find out which branch describes the bag. The checker requires both branches to reach that conclusion and keeps any other assumptions each branch used.

For an arbitrary object, the restriction is different. We may prove a property of an object without using anything special about it, then conclude that every object has the property. If an open assumption says $a \in b$, the particular name a is already doing special work. We cannot use that assumption to conclude that everything belongs to b. The name chosen for an arbitrary object must be absent from the assumptions still open.

We can assemble these larger moves from a small stock of signs. "If A, then B" rules out A together with the failure of B. "A or B" rules out their both failing. "Every x has property A" rules out an example lacking it:

$$A \to B := \neg(A \wedge \neg B),$$
$$A \vee B := \neg(\neg A \wedge \neg B),$$
$$\forall x\, A(x) := \neg \exists x\, \neg A(x).$$

For example, if a derivation takes us from an assumed A to B, anyone offering $A \wedge \neg B$ supplies both what starts that derivation and what contradicts its ending. The negation rule closes the attempt and

gives the first displayed formula. Once this implication and A are available, classical contradiction gives B. The "or" allows both alternatives to hold; a bag containing both a and c still has a member.

Appendix B follows a few of these arguments with all their assumptions left visible. The full collection of proof cards appears in Chapter 2 of the *Technical Companion*. Their basic logical signs are $\wedge$, $\neg$, $\exists$, with membership, equality, and the contradiction marker. The abbreviations let us use familiar forms of reasoning without asking the checker to acquire a separate faculty for each one.

We can now say exactly which sets our proof method promises. For collecting finite records we need an empty collection, and the ability to add one object to a collection already available. Adding that object is called *adjunction*. These two promises are Adjunctive Set Theory, or AS. Its first axiom says that an empty object exists:

$$\exists e\, \forall z\, \neg(z \in e). \qquad \text{(Empty)}$$

The second says that, given x and a, there is a b containing precisely the old members of x together with a:

$$\forall x\, \forall a\, \exists b\, \forall z\, \big(z \in b \leftrightarrow (z \in x \vee z = a)\big). \qquad \text{(Adjunction)}$$

The sign $\leftrightarrow$ means "if and only if," two implications joined by "and." The left-to-right implication keeps uninvited objects out; the right-to-left one puts the specified objects in.

Choose an empty object e and adjoin e to it, obtaining b. To see that $e \in b$, use the membership description for the particular object e:

$$e \in b \leftrightarrow (e \in e \vee e = e).$$

The equality $e = e$ gives the right-hand alternative, which gives the disjunction. The implication then gives the membership we wanted. These moves fit together as a proof tree:

$$\frac{(e \in e \vee e = e) \rightarrow e \in b \qquad \dfrac{e = e}{e \in e \vee e = e}\ \text{or}}{e \in b}\ \text{use if.}$$

Emptiness gives $\neg(e \in e)$. Our earlier contradiction argument therefore shows that $b \neq e$. We have made two objects whose difference is provable from the axioms. Adjoin b to b, obtaining c. Its old member e remains, and its new member b joins it:

$$\frac{e \in c \qquad b \in c}{(e \in c) \wedge (b \in c)}.$$

The old-member part of Adjunction supplies the first premise; the new-member part, with $b = b$, supplies the second. The rule for "and" has not changed since our first proof tree.

We can also start with an arbitrary object a and adjoin it to an empty object. The resulting collection contains a, so it supplies an example of a set to which a belongs. Close the temporary witness names for that collection and for the empty object. No open assumption then singles out a, and the arbitrary-object rule gives

$$\forall a \, \exists y \, (a \in y).$$

Every object belongs to some object. Nothing here promises one enormous object containing everything. Each a may have its own suitable y.

Nor do the two axioms say that collections with the same members must be the same object. That further principle is *extensionality*. We have not included it. We have also not assumed a completed infinite set. The axioms let us make the particular finite collections our records need, while leaving open what else a world satisfying them may contain.

Now try to preserve the word 707 in a set. Put 7 in, put 0 in, put 7 in again. We have lost both the order and the second occurrence of 7. The bag is being a perfectly good set and a poor notebook. Attach the position to each digit before collecting them instead:

$$(0, 7), \qquad (1, 0), \qquad (2, 7).$$

Each *ordered pair* has a first item, the position, and a second item, the mark. The two sevens now occur in different pairs. But if our only containers are unordered sets, we must explain how a pair keeps its first item first.

For two objects a, b, make a container u with just a, a container v with a, b, and a container p with those two containers:

$$u = \{a\}, \qquad v = \{a, b\}, \qquad p = \{u, v\}.$$

Empty and Adjunction supply the containers. The braces specify their members; they do not assert that there is only one object with those members. Open the inner containers of p. The object common to all of them is a, because the container u has no other member. Any other object that appears is b. If no other object appears, then $b = a$. The first and second items can therefore be recovered.

For $(0, 7)$, the inner contents are $\{0\}$ and $\{0, 7\}$: 0 is common, and 7 is the other item. For $(7, 0)$, they are $\{7\}$ and $\{7, 0\}$, recovering 7 first. For $(7, 7)$, only 7 appears in either inner container, recovering it twice. Distinct containers with identical contents cause no confusion about these recovered items. We need a reliable representation of the pair, not a unique physical bag for it.

Collecting the three positioned digits now preserves 707. We can treat a configuration similarly: join each mark to its address, each card name to its address, and the next instruction to its designated place. Give the configurations positions in the trace as well. The sequence of sequences can be recovered from the nested collections.

One check from r can be written out so that we can see what these containers are doing. In entry 1, let m name the collection of marked-square entries, a the address $(0, 0, 0)$, and s the code for 7. The assertion that the required address–mark pair occurs in m is

$$\begin{aligned}\exists t \exists u \exists v \big(& t \in m \wedge \\ & \forall z (z \in u \leftrightarrow z = a) \wedge \\ & \forall z (z \in v \leftrightarrow (z = a \vee z = s)) \wedge \\ & \forall z (z \in t \leftrightarrow (z = u \vee z = v))\big).\end{aligned}$$

Read upward from the inner containers if that is easier. The second line gives u just the address a; the third gives v the address and the mark. The last line puts u, v into t, making the pair code. The first line puts t into the marked-square collection m. Construct these objects by adjunction, and take m to contain the resulting t. They

supply the examples for the three “there exists” signs and a proof of the displayed claim.

A record with a 9 instead of a 7 fails the comparison with the distinct code for 7. A record with the correct mark but a moved address can fail a different check. We must also check the next instruction and every part that was supposed to remain unchanged. The statement above certifies one piece of a transition. It cannot excuse errors in the rest.

There is a further step between this example and a language for all computation records. We can build each particular finite record from finitely many containers. Our searcher needs one fixed description of what counts as a record, however long it is, and uniform ways of reading its entries. The small construction does not establish that larger result.

Here we use a coding theorem. Zlatan Damnjanovic proved that adjunctive set theory, with or without extensionality, and Robinson arithmetic, a weak arithmetic theory, can represent each other. Albert Visser’s work supplies the sequence-coding interpretation used for our records.[1]

An *interpretation* supplies formulas saying which objects represent numbers and records, how to read their entries, and when different codes represent the same thing. Recall what went wrong with 707: membership alone gave no positions. The interpreted arithmetic and sequence laws supply the required structure uniformly, with proofs in AS. We continue to use Empty and Adjunction as our axioms; the representation lets those axioms talk about more than their short statements suggest.

Fix that representation, including the formulas for instructions and output words. It has a property we will use immediately: every actual successful finite computation can be converted mechanically into a proof of its output sentence. We call this the *finite-trace property*. For r, the individual checks we have been making become pieces

[1] Damnjanovic, “Mutual Interpretability of Robinson Arithmetic and Adjunctive Set Theory with Extensionality,” *Bulletin of Symbolic Logic* 23 (2017), 381–404, doi:10.1017/bsl.2017.30. Chapter 2 of the *Technical Companion* specifies the arithmetic and sequence properties used here and gives Visser’s references. The full interpretation is not constructed in this book.

of such a proof. A longer completed record takes more pieces, but the conversion finishes for each record. The interpretation justifies this uniform conversion; our three-entry illustration alone did not.

Suppose our waiting program found a proof excluding 314, then responded by writing those digits and stopping. Its search up to that proof, its checking, and its final writes would make one completed finite computation. The finite-trace property would turn that computation into a proof that it returned 314. The response would therefore furnish a second proof, contradicting the very claim that set it going. We now have the means to make the program's provocation exact.

The waiting program wants to use proofs without waiting for a mathematician to invent them. It can do so in the most extravagant way imaginable: try every finite piece of writing. Order the strings first by length and then alphabetically. With a two-symbol alphabet the list begins

$$0,\ 1;\qquad 00,\ 01,\ 10,\ 11;\qquad 000,\ 001,\ 010,\ldots.$$

Use our proof alphabet in place of the two symbols. Names such as `v0` and `v17` provide indefinitely many variables using finitely many marks. Any particular finite proof has a place in this list, with only finitely many strings before it.

For each candidate, the checker works from the leaves of the proposed tree toward its conclusion. It matches the rule at each step, checks substitutions and fresh names, and carries the remaining assumptions forward. From $e \in c$ and $b \in c$, the conclusion $(e \in c) \wedge (c \in b)$ fails: the second pair of names has been reversed. Intending the right statement cannot make the written step pass.

Most candidates are not even grammatical. Of the grammatical ones, most prove nothing. The search goes on with a dedication no writer should emulate. Each check finishes, and every valid finite proof eventually gets checked. If the desired conclusion has a proof, the search finds it. If it has none, the search continues. Looking for both a statement and its negation does not guarantee an answer; both searches can remain unsuccessful.

We can allow more axioms than the two we have printed, including an endless list produced by a program. A proof using an axiom

from that list attaches a finite record of the axiom-printer reaching it. The checker examines the attachment rather than waiting to see what the printer might produce in the future. Thus each candidate still receives a finite check, and every genuine certificate remains eventually available to the search.

Our contrary program can now have exact instructions. Call the chosen proof method T. Before constructing the program, fix how a claim about an output is written. Given program text e and a finite digit word w, the sentence-making procedure produces Out(e, w): there is a correct completed record of that program ending with that word. The procedure must finish and must describe ordinary finite records correctly. Its written form must also let the searcher recover w from a matching conclusion. Recognizing a claim about 314 has to reveal the three digits to be copied.

For the argument, T must use our classical first-order proof rules, its certificates must be mechanically enumerable, and the fixed output sentences must have the finite-trace property in T. Assume also that T is *consistent*: it never proves both a statement and its negation. AS with the fixed interpretation is our intended example. These are actual requirements, not consequences of calling a method consistent. A consistent collection of writing that has no way to describe computations would give this searcher nothing suitable to recognize.

Let the program search for a proof of $\neg$ Out(e, w), for any finite digit word w. When it finds one, it copies w to its result and halts. Kleene supplies its own complete description in place of e. With `SELF` naming that description, the task has this short form:

```
Search: Call NextProof(proof)
Call ExcludedWord(proof, SELF, answer, word)
If(answer,N) Goto Search
Call Copy(word, RESULT)
Halt
```

`NextProof` continues the enumeration until another checked proof appears. `ExcludedWord` looks for the specified negative output claim about `SELF`. A match puts Y in `answer` and the excluded word in `word`; otherwise it puts N in `answer`. The program recog-

nizes a written pattern. It is not required to agree with what the pattern says.

Call this complete program E. Suppose a matching proof exists. There is a first one in the search order; say it excludes 314. The program reaches it, recognizes the claim, copies 314, and stops. This is a particular finite computation, including all the searching needed to reach that proof. By the finite-trace property, it yields a T-proof of $\mathrm{Out}(E, 314)$.

But the proof that prompted the response was a T-proof of $\neg\,\mathrm{Out}(E, 314)$. We have obtained both a statement and its negation from a method assumed consistent. The supposed first proof therefore cannot exist. If it had excluded 0007 instead, the program would have copied all four symbols and supplied the corresponding contradiction. Here outputs are words: the zeros in 0007 are part of the promised response.

For every particular finite word w, we have shown

$$T \text{ does not prove } \neg\,\mathrm{Out}(E, w).$$

In the ordinary execution, E searches forever. There is no matching certificate, and finding one is its only route to an output. Its result row stays blank. Yet T cannot exclude any specified finite result. It cannot rule out 314, or 0007, or the word we might choose tomorrow.

The conclusion depends on only one direction of trust between computation and proof. When a computation actually finishes, its finite record can be made into a proof. We have not assumed that every proof asserting a completed computation gives us an ordinary finite record that can actually be produced. Consistency forbids contradictory theorems. It does not, by itself, make all theorems about the worker true of the work at our desk.

Try adding $\mathrm{Out}(E, 314)$ to T as an extra assertion. If this caused a contradiction, take the contradicting proof and close the extra assumption by our negation rule. That would leave an old T-proof of $\neg\,\mathrm{Out}(E, 314)$. We have just shown that no such proof exists. The new assertion can therefore be added consistently.

The same holds for any particular finite word, considered separately. Choosing 314 and choosing 0007 give different extensions of

the proof method. Nothing in the argument permits us to demand both incompatible results at once. Later we will want something more carefully coordinated: a longer decimal beginning that keeps the shorter beginning already chosen. For now, even this single consistent assertion of 314 is strange enough. The program it describes is still searching.

We can isolate the failure of proof with an even smaller searcher. Let J look only for a proof saying that J never halts, and halt if it finds one. Kleene again ensures that the sentence concerns the complete program that responds. There is no other exit. If the proof existed, the program would reach it and halt; its completed trace would then supply a proof that it halts. Consistency excludes the pair, so the searched-for proof does not exist.

We can finish that argument from outside the search. With no proof to find and no other way to stop, J runs forever. The sentence it seeks is true and unprovable in T. This does not show, from consistency alone, that its opposite is unprovable. A consistent theory need not be correct about ordinary computations. Rosser's refinement obtains a sentence with neither side provable under the appropriate hypotheses; the separate theorem about proving a theory's own consistency is another result. Neither is needed for the consistent assertion about 314 that is now on our desk.

Gödel first revealed the incompleteness obstruction. Our route through Turing's worker and Kleene's self-description came later. Hilbert's requirement that proof be exact enough to check has allowed a program to read a proof of its own behaviour, and Church's world of expressions has supplied the means of making that behaviour answer back. The rules did not become loose. They became precise enough to support the construction that exposes their limit.

Leave E running. Beside it put the consistently enlarged theory, with its assertion that E returns 314. We know what a completed record would have to contain: the prescribed beginning, every instructed change, and the final writes. No ordinary completed record lies between the two pieces of paper. The argument has secured consistency without filling that gap. Henkin's construction will begin by giving the promised record a name, and then build a world in which that name has something to name.

Chapter 6

A world for the words

Writing 314 takes three strokes of the pen, give or take one's handwriting. Supplying the computation that puts it there is another matter. Our searcher E has been told to find a proof forbidding an output and then produce that output. Consistency keeps every such proof out of its reach. The search continues; the answer row stays empty. Yet the sentence "E finishes with 314" can be added to our assumptions without producing a contradiction.

We cannot settle this by writing the answer for it. A checker would want the preceding configuration, and the one before that, all the way back to the blank starting page. A declaration of success has incurred a debt of steps. Each must be accounted for, under the very instructions that keep our worker searching.

Leon Henkin found a way to start with consistent declarations and build a setting that fulfills them. He did not begin by finding the objects his sentences mentioned. He began by giving them names. Henkin was another of Church's doctoral students; his doctorate came in 1947, and his completeness proof appeared in 1949. Gödel had already proved completeness. Henkin's construction gave the result a particularly tangible form: the writing itself could supply the materials of a world. In his later account he credited Church's teaching and writing with the main ideas behind his discovery.

That is a considerable assignment for some ink. Before asking it for the missing computation, let us see what it can do with three letters. Suppose a consistent notebook permits these entries:

$$a \in b, \qquad a \neq b.$$

Something named a belongs to something named b, and they are different objects. An empty set and the set containing it would meet these two demands. We need not choose those particular objects; for the moment the notebook records only what any eventual choice must respect.

The first entry also says that b has a member:

$$\exists x\,(x \in b).$$

Give that promised member a new name, c, and add $c \in b$. Then ask whether $c = a$. In this example we may answer yes. The new name can name the member we already had. We have used another letter without multiplying the population.

Here is a little world that the entries are beginning to describe:

$$\underbrace{\{a, c\}}_{\text{names of one object}} \xrightarrow{\text{belongs to}} \underbrace{\{b\}}_{\text{names of another object}} .$$

Call the left object U, the right one V. The arrow gives U a place in V. It answers the claim $a \in b$ and the claim $c \in b$ at once, and supplies the member promised by "there is." The words have begun to acquire things they are true of.

They have not yet supplied a whole set-theoretic world. We have only a fragment, and even that fragment leaves questions open:

belongs to	U, named by a, c	V, named by b
U, named by a, c	undecided	yes
V, named by b	undecided	undecided

An undecided square is not a no. More writing must settle it. And there will be many more names: the axioms demand further sets, whose members in turn need places in the picture. What makes Henkin's construction work is a way of continuing this small exercise without losing consistency or leaving an existence promise unfulfilled.

The new name matters. Before we introduced c, it had no commitments in the notebook. An old name might already have been ruled out as a member of the set in question; forcing that name to serve as the promised member would manufacture a contradiction. A fresh name avoids this trouble. It need not denote a new object, as $c = a$ just showed. It needs to arrive without a past in the notebook.

We can prepare for an existence claim even before deciding whether it is true. Our notebook already uses c, so reserve an unused name, e, for the next promise. For a condition $A(x)$, enter

$$(\exists x\, A(x)) \longrightarrow A(e).$$

We could take $A(x)$ to be $x \in a$, asking this time whether a has a member. The entry says: if anything meets the condition, e will name an example. If nothing does, it demands nothing of e. This is a permanent naming arrangement. The temporary witness in a proof branch disappeared when the branch closed; this name stays available for later conversation.

Why is that arrangement safe? Suppose the new promise about membership in a produced a contradiction. Close the assumption containing the promise, using our proof-by-contradiction rule. The old notebook would then prove that the promise was false. An "if" fails only when its first part holds and its second part fails, so the old notebook would prove both

$$\exists x\,(x \in a) \qquad \text{and} \qquad \neg(e \in a).$$

But e is absent from every remaining assumption. Nothing in that proof of nonmembership could depend on which object e named. Our arbitrary-example rule therefore gives $\forall x\, \neg(x \in a)$: nobody belongs to a. The existence claim lets us temporarily call one member d. We obtain $d \in a$ and $\neg(d \in a)$, and close the temporary branch with their contradiction. The old notebook could already contradict itself. If it was consistent, the supposed trouble from our new promise cannot occur.

Nothing in this reasoning depended on membership being the condition. Replace $x \in a$ by $A(x)$ and the same proof works. The freshness restriction from our proof cards has earned its keep: once the trial assumption is gone, the remaining proof treats e as an arbitrary object.

We must also finish the undecided squares, and eventually every other question the language can ask. Does V belong to U? Try the two possible additions, $b \in a$ and $\neg(b \in a)$. At least one preserves consistency. If the first caused a contradiction, the rejection rule

would give its negation from the old notebook. If the second also caused a contradiction, classical reductio would give the positive claim. The two failures would already be a contradiction in the old notebook. The same argument applies to any sentence, however elaborate its writing.

This gives us somewhere to go. It does not give the worker a reliable way to tell which way. Looking for a contradiction can reveal one; failing to find it so far is still only failing to find it so far. The construction may require choices no instruction sheet can make. We encountered this gap in the decimal game: proving that room exists for a move is different from programming every move.

Call our starting collection of assumptions T. Keep a numbered reserve of new names, none appearing in T, and arrange two lists of jobs: decide a sentence; provide a fresh-name promise for an existence statement. Alternate between the lists. All finite expressions in the enlarged language can be listed, so we include sentences involving the new names as well as the old ones. At a decision, take a consistent choice. At a promise, choose a name unused in the work so far and absent from the formula being considered. Even if T is infinite, it uses none of the reserve names, and only finitely many have been used at any finite stage. We cannot run out of names before the next job.

Every stage is consistent. Could the whole endless notebook somehow become contradictory? A proof of contradiction would be a finite tree. Circle the entries it uses. There are only finitely many, so all were already present at some finite stage. We would have had a contradiction there. An infinite supply of pages does not give a finite proof an infinite supply of premises.

Include all the notebook's provable consequences and call the result K. Each sentence is decided, and each existence claim has a name ready for an example. We have reached this object by an existence argument, not by finishing the worker's to-do list. Now we can extend the construction of U and V to all its names.

Put together every pair of names that K declares equal. Equality rules keep the grouping in order: a name equals itself; an equality can be reversed; and if $a = c$ and $c = e$, then $a = e$. The groups are called *equivalence classes*. Make one object for each group. All the

names in it will refer to that object. Our earlier a and c must stay together, while $a \neq b$ keeps b apart from them.

For each pair of groups, use the notebook to fill its square in the membership table. An entry $a \in b$ puts yes in the square; $\neg(a \in b)$ puts no. Because K decides the sentence consistently, exactly one answer is available. Changing from a to an equal name c changes nothing: equality substitution requires the same answer for $c \in b$. The relation belongs to the objects, regardless of which of their names we use.

There are two different jobs for membership here. The letter a is one of the names collected into U. The table says whether U belongs to V in our constructed world. Looking inside the braces around $\{b\}$ will not answer that second question. Those braces help us make the objects; the table supplies the relation the notebook calls membership.

A *model* is such a setting of objects and relations in which all our starting sentences hold together. We have made its objects from names and its membership from the entries in K. The remaining question is whether whole sentences, with their "and," "not," and "there is," agree with the world as well. A table of membership facts would be rather a small reward for all that naming.

Take $(a \in b) \wedge (c \in b)$. The notebook contains it exactly when it contains both parts: the conjunction rules let us assemble it or take it apart. Our table makes it true exactly when both squares say yes. Thus the notebook and the model give the same answer to the joined claim. For a negation, K contains exactly one of A and $\neg A$. Once A agrees with the model, its negation agrees too: it is in the notebook just when A is false there.

Existence was the reason for our fresh names. If K contains $\exists x\,(x \in b)$, the promise gives $c \in b$. The group named by c is an actual example in the table. For a general condition $A(x)$, its reserved witness name supplies an example in just the same way. The argument also runs back toward the notebook. Suppose an object in the model meets A. Every object has a name; choose one of its names, t. Once the smaller statement $A(t)$ agrees with the model, it must be in K. The proof rule for giving an example puts $\exists x\, A(x)$ in K as well.

There is no circular appeal to the truth of the whole sentence here. Its construction is finite. Start with the equality and membership claims, already checked, then work outward through each "and," "not," and "there is." A layer agrees because its smaller pieces agree. This is the *truth lemma*: a sentence is in K exactly when it is true in the constructed world. Our abbreviations for "or," "if," and "every" use the same three operations, so they come along too.

The quantifiers range over the objects we made. In $\exists x(x \in b)$, we ask whether one of those objects is a member of the object named by b. "Every" asks about every object in that collection. This is the *first-order* language we have been using. A set may itself be one of its objects, but the quantifiers do not thereby reach every collection that someone outside the model can imagine.

The names were on a numbered list. Replacing each name by its group lists the model's objects, perhaps several times each. From outside, then, this is a countable model. Its own lists, if its axioms describe lists, are among its objects; our outside numbering need not be one of them. This is the distinction behind Skolem's observation, which we follow further in Chapter 4 of the *Technical Companion*.

Every assumption in T belongs to K, so the truth lemma makes all of them true in our model. We have reached Henkin's model-existence theorem:

> Every consistent first-order collection of assumptions
> in a countable language has a model.

Completeness in this setting connects proof with interpretation. It is a different use of the word from the completeness of the real line that kept Baire's nested intervals from losing their point.

Return now to the debt of steps behind 314. A useful consequence of Henkin's theorem comes from the same finite proof tree we used to protect the notebook. Suppose every finite selection from a countable collection of requests can be met in some model. If the whole collection proved a contradiction, that proof would use only a finite selection. In the model meeting those requests, every assumption of the proof would be true. Valid proof steps preserve truth; a contradiction cannot be their result. Thus the full collection

is consistent, and Henkin supplies a model for it. This is the *compactness theorem*. The models for the finite selections can differ. What matters is that no finite proof can collect enough premises to cause trouble.

Try a request that becomes harder as we keep making it. Let b name a natural number, and require

$$b > 0, \qquad b > 1, \qquad b > 2, \qquad b > 3, \qquad \ldots.$$

For this example include the true first-order statements of ordinary natural-number arithmetic, with its order, addition, and multiplication. Any finite selection of the new demands has a largest numeral on the right. Choose b larger than that numeral and use the ordinary natural numbers as the model. All the selected demands hold. Compactness therefore gives a model in which the entire list holds for one b.

Where could that number have come from? We have not asked for b to exceed every number in the model. That would include b itself, demanding $b > b$. We asked separately that it exceed zero, one, two, and every other *ordinary numeral*. The model has objects named by those numerals and also objects beyond them. Its b is one of the latter, a *nonstandard* number. Its arithmetic still supplies $b + 1$, which is larger.

Nor is there a first nonstandard number just after the ordinary ones. Every positive number has a predecessor under the arithmetic laws we included. Given any ordinary numeral n, the list requires $b > n + 1$, and hence $b - 1 > n$. So $b - 1$ also exceeds every ordinary numeral. The same goes for $b - 2$ and $b - 3$. There is no last ordinary number to stand on one side of a border, and no first extra number to stand on the other.

The notation can make this look deceptively accessible. Write

$$0, 1, 2, 3, \ldots \qquad\qquad \ldots, b - 2, b - 1, b, b + 1, \ldots.$$

Our eyes cross the gap in a moment. They have crossed some blank paper. The display offers two small views of the order; it has given us no ordinary sequence of counting steps that travels from one view to the other. Every ordinary count we finish is still followed by another ordinary count.

A record that a model calls finite can have a length like b. It has a last line according to the model, although its length exceeds every ordinary numeral. Calling that line the last does not get our worker there. This is how the missing computation can be supplied without an ordinary search ever finishing.

Chapter 5 proved that our assumptions together with "E outputs 314" are consistent. Henkin therefore gives a model in which the output sentence holds. The sentence asks for a completed computation record, and the model has one. It must still obey our fixed interpretation of instructions and records. Those laws prescribe the decoded starting configuration and the unique decoded successor at each nonhalting instruction: the specified mark changes, or the specified place card moves, while everything else stays put.

These requirements are stronger than the earlier promise that an actual successful calculation can be proved successful. They also control what the formulas say across models. Chapter 2 of the *Technical Companion* sets out the required arithmetic and record laws. Apply them once: the model's next configuration agrees with the actual worker's. Apply them again: so does the next. Repeat the argument for any ordinary finite number of steps. Every one agrees, and none is a halt. The completed record in the model must end beyond every ordinary stage.

The model cannot merely announce 314 at that remote last line. The search has one way out, so its record includes what it accepts as an exclusion certificate, the check of that certificate, and the return of the forbidden digits. An ordinary finite certificate would be disastrous: our worker could reach it, respond with the digits, and supply the opposite proof. That is exactly the contradiction consistency ruled out. The model's certificate cannot be brought back as an ordinary finite proof for us to check.

The objects over which "there is a finite record" ranges have changed. The instruction at an ordinary step has not. We can now let two searches stand beside the old division sheet, one to supply its numerator and one its denominator. Their results and the division will have to stay together in a single record. Then each new digit we ask for can place another demand on that same calculation. The notebook is ready; the division has some requests to make.

Chapter 7

Long division again

The division sheet has been waiting for us. Its rules still say to bring down a zero, subtract the divisor as many times as possible, and write the number of successful subtractions. Nothing in the intervening chapters has taught a remainder to behave differently. What has become less straightforward is getting the two integers onto the page.

For the moment, we can supply them ourselves. Suppose we want the decimal to begin with 314. The drawer runs from 0.314 to 0.315. Halfway across sits

$$0.3145 = \frac{629}{2000}.$$

Here is the old procedure at work on those integers:

place	ten times old remainder	digit	new remainder
1	6290	3	290
2	2900	1	900
3	9000	4	1000
4	10000	5	0

The first line subtracts three copies of 2000 from 6290. The 290 left over becomes 2900 on the next line, where one copy fits. Then four copies fit, then five. We have our 314, followed by a five and a remainder of zero. More steps would write zeros.

We could have asked for 003. Taking the midpoint again gives $0.0035 = 7/2000$. A numerator of seven has no zeros at its front, but dividing by 2000 makes the worker write them. Three places were requested, and the denominator remembers how many. For a word of k digits whose integer value is a, the same recipe makes

$$\frac{2a+1}{2 \cdot 10^k} = \frac{a}{10^k} + \frac{1}{2 \cdot 10^k}.$$

The term on the left of the plus sign is the drawer's lower end; the other term moves halfway across. The worker doubles a, adds one, and divides by a two followed by k zeros. The answer begins with the requested word, then a five, then zeros.

Even 000 and 999 cause no difficulty:

$$\frac{1}{2000} = 0.0005, \qquad \frac{1999}{2000} = 0.9995.$$

A finite beginning is easily accommodated. The difficulty we have been pursuing belongs to the whole endless writing. We defeated every ordinary fraction in the decimal game, then every ordinary digit-printing program. Yet here is division, willing to reproduce whatever finite evidence of our victory we put in front of it.

So far the requested word has chosen the numerator and denominator. We will reverse the arrangement. Two instruction sheets, P and Q, will be fixed before any infinite decimal is chosen. Their job is to supply the two integers. They get their material from an unlikely source: a proof that their division cannot begin with some finite word. Give them an objection to 314 and they will prepare 629 and 2000. The objection has included the very digits needed to answer it.

We have met this response before in E. What is new is the need to keep a whole calculation together. A claim that one program has an output, another has an output, and some fraction somewhere begins with 314 would leave too much room between the pieces. The division must use those outputs, and a later request must refer to that same division.

Take the completed work for 629 and 2000. Alongside the two printer records, save how their written numerals acquire their values. Reading 629 from left to right gives

$$0 \longrightarrow 6 \longrightarrow 62 \longrightarrow 629.$$

At each arrow the value so far is multiplied by ten and the next digit added. The denominator's chain is $0, 2, 20, 200, 2000$. Let p and q denote the last values. The record requires $q > 0$ and $0 \le p < q$, and attaches a division of precisely these integers, starting with 0..

The division table retains the connection to its numerator at every line. For our example the checks are

$$\begin{aligned} 10 \cdot 629 &= 3 \cdot 2000 + 290, \\ 100 \cdot 629 &= 31 \cdot 2000 + 900, \\ 1000 \cdot 629 &= 314 \cdot 2000 + 1000, \\ 10000 \cdot 629 &= 3145 \cdot 2000 + 0. \end{aligned}$$

The remainders remain nonnegative and smaller than 2000. This is Chapter 2's invariant, carrying the original division through its changing answer row. The third line puts $1000 \cdot 629$ between $314 \cdot 2000$ and $315 \cdot 2000$, so

$$0.314 < \frac{p}{q} < 0.315.$$

The first two lines similarly place the very same quotient in the drawers for 3 and 31. We have not changed fractions as the answer became more specific.

Call the joined records a packet, and give the packet the name d. Its checks include the two completed runs, the values of their outputs, and a division record ending at a digit boundary with remainder zero. It may keep further zero-writing steps after the first zero remainder. For a finite word s, abbreviate the claim we want by

$$\Phi_s = \exists d \, \big(\mathrm{Record}(d) \wedge \mathrm{Prefix}_s(d)\big).$$

It says that one such packet exists and its division starts with s. The prefix part checks the digits and the interval inequalities for that same p, q. It also requires enough recorded places. Asking for a thousand zeros demands a record of a thousand places, even though the answer row would be easy to describe in fewer words.

We arrange these requirements so that a longer word keeps the earlier checks. The request for 3141 includes the request for 314, then adds a fourth digit and the narrower interval. Our packet for 629/2000 fails the addition: its fourth digit is five. Any packet that passes, however, still passes every earlier check. The formulas join the old conditions to the new ones with "and," so our proof rule

can take the old conditions back out. This small feature will keep one calculation in view when the requests become endless.

Our proof method T and its fixed record coding must be able to handle this packet. We retain the assumptions from Chapter 5: T is consistent, its certificates can be mechanically enumerated, and actual finite successful work can be translated into proofs of the corresponding record claims. Here that work includes the *combined* calculation, not just two isolated outputs. Across models the coding must also retain the stated ordered arithmetic and the deterministic changes of configuration. These are requirements on the interpretation used in our adjunctive setting, set out in Chapter 2 of the *Technical Companion*; consistency by itself does not give an arbitrary theory this ability to talk about computation. We will use the requirements at the two places where they are needed: to verify a hypothetical ordinary answer, and to understand a model's extraordinary one.

The search can now have a definite target. It lists finite words beside candidate certificates, checks the certificates, and looks for a conclusion of the form $\neg\Phi_s$. Such a conclusion forbids a common packet with the beginning s. When the search finds one, it returns the word. The numerator worker doubles its value and adds one; the denominator worker writes a two followed by as many zeros as the word has places. Both follow the same ordered search from empty input.

One knot remains. The sentences being searched for must concern these very workers. Kleene's copying construction lets us tie it without asking either worker to recognize itself in a mirror. Recall the white and blue sheets: a supplied text was copied into a package, and the other sheet supplied the text that made the package reconstruct its own description. The task could then use that description as ordinary input.

For our present task, take a proposed controller text e. Write a description P_e saying to obey e, take its returned word, and perform the numerator job. Write Q_e with the denominator job instead. Preparing these descriptions requires copying instructions, not waiting for them to finish. Using the two descriptions, the task writes the formulas for their common records and starts the search

for exclusions. Kleene supplies a controller e carrying out this task on its own description. The workers it describes therefore begin by obeying the controller that has just described them.

Follow the response once, assuming a certificate arrives. The worker on P_e runs the controller; the controller reconstructs the printer descriptions and searches for an exclusion. Suppose the first excludes 003. It returns that word, and P_e calculates $2 \cdot 3 + 1 = 7$. The identical search inside Q_e returns 003 too, so Q_e counts its three places and writes 2000. The quotient is 0.0035, as required. The copying construction has supplied descriptions of this behavior without presuming that a certificate ever arrives.

Fix the resulting sheets and call them P, Q. They depend on the proof method T, whose checker they use. They do not depend on any chosen infinite decimal.

Now suppose an exclusion certificate exists. Take the first the search would find; say it excludes 314. Reaching that particular certificate takes an ordinary finite amount of work. The check, the return of the word, the printing of 629 and 2000, and the four division lines also take ordinary finite work. Together they give exactly the packet above. Our verification assumption turns it into a T-proof of Φ_{314}.

The certificate was a T-proof of $\neg\Phi_{314}$. Consistency forbids the pair. Nothing depended on those three digits: whichever finite word appeared first, the response would build its midpoint fraction and verify a packet with that beginning. There is therefore no ordinary exclusion certificate for any finite word. Our printers remain in their searches forever. We have designed two integer printers whose answer rows stay blank, and the proof of their usefulness has begun by establishing this defect.

For each particular s, the absence of an exclusion proof also tells us that $T + \Phi_s$ is consistent. If this extra assumption led to a contradiction, closing the assumption would give a proof of $\neg\Phi_s$ in T, the certificate we just ruled out. Henkin consequently supplies a model of $T + \Phi_s$, with a packet realizing that finite beginning. We have models for 314, for 3141, and for any finite continuation we care to name.

Separate models still leave us short of the promised decimal. One model could supply three places, another four; each new digit would send us shopping for another calculation. Instead choose an entire infinite digit sequence, perhaps beginning 31415926 . . ., and give the packet a single new name, d. The new notebook contains T, the assertion that d is a checked packet, and these requests:

$$\mathrm{Prefix}_3(d), \quad \mathrm{Prefix}_{31}(d), \quad \mathrm{Prefix}_{314}(d), \quad \mathrm{Prefix}_{3141}(d), \quad \ldots.$$

Each entry uses the same name. The list runs through every ordinary finite beginning of the chosen sequence.

Take any finite selection of these entries. It has a longest requested beginning. We already have a model of T with a packet for that word; interpret d as that packet. Because its prefix checks include all the shorter ones, this one packet meets every request in the selection. It also meets the common-record requirements. Thus every finite selection can be satisfied together.

Compactness gives one model satisfying the entire notebook. Its packet contains one pair of completed printer runs, one pair of integer values p, q, and their division. Every ordinary finite request applies to that same packet. This is where the fastening of the records pays for itself: the fractions cannot change between one digit and the next.

The order of our choices is part of the result:

> Fix a consistent, mechanically enumerable classical first-order theory T and coding with the arithmetic, deterministic-record, and finite-verification properties described above. There are two fixed programs P, Q such that, for every external infinite decimal sequence, some model of T contains one completed record of their integer outputs and their division, agreeing with that sequence at every ordinary finite position.

The programs are fixed before the sequence. The model may depend on the sequence, and need not extend a particular model we already have. An uncomputable sequence supplies no algorithm for writing all the requests into Henkin's notebook; the sequence

need not even be one of the eventual model's objects. We used its finite beginnings to prove the existence of a suitable setting.

The quotient is rational in that setting. Our first escape in the decimal game was from rational numbers. It is worth staying with this apparent collision until we can locate it on the division sheet.

The packet has a last saved line, with remainder zero. Suppose it were line one thousand. Our notebook also asks that the same packet contain the first thousand and one digits. Suppose it were line one million; a longer request would reach past that too. The last line's index, b, must exceed every ordinary numeral. Inside the model this is a finite record that reaches zero and would thereafter print zeros. None of its ordinary positions has to be where that final tail begins.

Choose the familiar growing-gap decimal:

$$0.01001000100001000001\ldots.$$

At every ordinary position the model's division agrees with it. There is always a later ordinary one, and another longer gap. The zero tail lies beyond all those positions. There is no first extra position immediately after ordinary counting, just as there was no first nonstandard integer in the last chapter. If $b-1$ were an ordinary position, then so would b be; the same reasoning holds for $b-2$, $b-3$, and each ordinary distance we count backward. The record's internal end does not furnish an ordinary place at which our target begins to repeat.

The printer records have the same separation from ordinary time. Their starting configurations are fixed, and their decoded successors are prescribed by the instructions. At any ordinary stage, repeating those checks identifies exactly the search we could perform at the desk. Neither search halts there. A model's completed record includes an internally accepted exclusion certificate; if that were an ordinary finite certificate we could check, we would recover the contradiction above. No new ordinary answer has appeared on either worker's paper.

The nines give the difference a numerical shape. Ask successively for 9, for 99, for 999. Our midpoint fractions are

$$\frac{19}{20} = 0.95, \qquad \frac{199}{200} = 0.995, \qquad \frac{1999}{2000} = 0.9995.$$

Each sits below one, with gaps $1/20$, $1/200$, and $1/2000$. Put all the nine-requests into the notebook and use compactness as before. Its common quotient still satisfies $p/q < 1$, because the record requires $p < q$, but every ordinary position is a nine.

Let $h = 1 - p/q$ be its positive gap. The request for $n + 1$ nines puts the quotient at least as high as $1 - 10^{-(n+1)}$, so

$$0 < h \leq \frac{1}{10^{n+1}} < \frac{1}{10^n}$$

for every ordinary positive integer n. A gap smaller than each of these ordinary decimal scales can exist in the model's ordered arithmetic. It is not a positive ordinary real number. Two distinct ordinary reals have some decimal scale fine enough to separate them.

Thus $0.999\ldots = 1$ remains exactly right when those nines name an ordinary real. The quotient in our model lives in a larger ordered setting; agreement at every ordinary decimal place does not identify an element of that setting. We may likewise request $0.3000\ldots$ or $0.2999\ldots$, the two ordinary names for the same endpoint. Both writings are allowed targets, possibly realized in different models. The promise concerns the digits at ordinary positions.

Now use the halting decimal from Chapter 4. Its positions answer our numbered questions about programs and inputs: does this run stop? We proved that no ordinary program prints all those answers. The same fixed P, Q nevertheless have a suitable model with one division record agreeing with each answer at its ordinary position. Its model need not call that record its own halting sequence. The target answers were chosen outside it. At our desk, the searches still do not finish.

The certificate reversal adapts the argument Hamkins attributes to Vadim Kosoy. Keeping the outputs and quotient in a common record lets the finite-prefix claims pass together into compactness.

The source notes distinguish this suitable-model result from the stronger extension results known for universal algorithms. Our claim remains conditional on the stated arithmetic and record-verification properties.

We began with a person who could do a calculation without understanding what it meant. That separation made the work checkable: a borrowed one belonged in its column regardless of anyone's opinion about numbers. Every ordinary step retains that exactness. But the claim that *all* the steps form a completed finite computation brings the surrounding world into the sentence. The instructions fit on their familiar pages. Whether those pages have a finished answer is no longer a question the pages can settle alone.

Coda: A new decimal in an old world

Ten digits fit on a small card. A hundred need more room. Whatever finite beginning we write, its ingredients are the same ten marks we brought from school. There is another way a mathematical world can change: every finished card can already be there, while the whole endless writing is new.

This time we keep the natural numbers fixed. An old program keeps its ordinary steps, and if it halts, its stopping time and answer. The new arrival will be an entire set of digits. But sets have company. A set of digits can belong to other sets; functions can take it as an input; those functions belong to further sets. To admit the decimal while preserving the axioms, we must make room for its mathematical relations too. Paul Cohen's method of *forcing* constructs such an extension.

For this glimpse, assume a world M satisfying the usual set axioms with choice. From outside, assume its objects can be listed. Also assume it is *transitive*: its sets have their ordinary members, and those members are themselves in M. Its natural numbers are then the ordinary ones. This starting assumption is stronger than the consistency of our adjunctive theory. Henkin gives countable models, but does not promise transitive ones.

The outside list does not contradict M's assertion that its reals cannot be listed. That assertion concerns lists among M's objects. Our outside numbering need not be available there. It will let us take the old world's requirements one at a time, as we took the obstacles in Baire's game.

Use just zero and one in the decimal places. A finite beginning is called a *condition*. Once we have written 010, we may continue to 01011; we may not go back and make it 011. Every extension keeps the decisions already made. Against an old sequence beginning 01001 . . ., for example, the condition 01000 has secured a permanent disagreement.

From any beginning we can arrange such a disagreement with any one old sequence. If they differ already, the work is done. Otherwise append the opposite of its next digit. The collection of conditions that have achieved the disagreement is *dense*: every condition has an extension in it. The requirement to write at least seven digits gives another dense collection, and so does each longer length requirement.

List all the dense collections of conditions that are objects of M. Extend a beginning to enter the first collection, extend again to enter the second, and continue. A few stages could look like

$$01 \quad \longrightarrow \quad 0100 \quad \longrightarrow \quad 01000 \quad \longrightarrow \quad 0100011.$$

The middle move disagrees with 01001 . . .; the last reaches seven places without undoing that disagreement. A general dense collection need not contain every later extension of a condition in it. We only need to have entered it once: that accepted prefix stays in our history.

Length requirements ensure that these compatible beginnings form an infinite sequence g. Write G for the collection of all its finite prefixes. Because G meets every dense collection belonging to M, it is called *generic over* M. The qualification matters. We have met the old world's dense collections, not every demand someone outside might invent. Nor has the outside enumeration supplied a program for making the choices.

For each old zero–one sequence, the collection requiring a disagreement belongs to M and is dense. Our g therefore differs from every old sequence. Its real value is new as well: two different zero–one decimals cannot be the two writings of a single real, since that duplication needs a tail of nines. If the value were old, its decimal expansion without an eventual nine-tail would be old too, and would have to be g. That would undo one of our disagreements.

The old world can prepare a name for an object whose value depends on these choices. Consider the following two-row card:

include this object	when this condition is accepted
zero	the first digit is zero
one	the first digit is one

A first digit of zero gives the card the value {0}; a first digit of one gives it {1}. Before either choice, the card already guarantees a set with a member. We know that much about its eventual value without settling which object will be the member.

This is a small forcing *name*: conditional directions for membership. A larger name says to include the old position n whenever an accepted prefix writes one at that position. It will name the set of one-positions in g. Names may refer to further names, allowing sets of the new objects and functions involving them. Old objects have fixed names that continue to denote those same objects. Some names contain infinitely many instructions; giving a name does not promise that a worker can finish evaluating it.

Evaluate all the names in M according to the accepted prefixes. Their values form the extension $M[G]$. The forcing theorem connects truth about those values with what conditions guarantee. A preservation theorem establishes that the extension still satisfies the set axioms. Our little card explains a membership choice; these general results do the work of constructing a whole world around the new decimal.

We can also recognize Henkin's notebook here: accepted conditions settle sentences about names, and the completed notebook supplies objects by grouping names declared equal. The forcing truth lemma connects this reading with the evaluated names. Chapter 4 of the *Technical Companion* follows that connection and the witness constructions it requires.

No extra natural-number times have appeared in this extension. An old computation has the same configurations at each step. An old program that never halted has no new ordinary occasion on which to halt. Giving it the new sequence as a source of answers would change its information; the unchanged instruction sheet has learned nothing. Our two integer printers, in particular, continue their ordinary searches.

Now write ten digits of g on a card. This finite string was already in M. Write a hundred, or any other ordinary finite number: that string was there too. Every card we finish contains an old object. Yet no old sequence contains all these beginnings in the required order, and no old instruction sheet prints the new whole. A zero looks just as it did before. The change lies beyond every finished card, in which endless writings the world contains.

Appendix A

The arithmetic sheets

Seven and eight make fifteen. In the addition 587 + 468, we put down five and carry one. The next column makes fifteen too; the last makes ten. The answer is 1055. Our hand can finish this small journey while a description of it is still putting on its shoes. Here we spread the journey out, so that each instruction can be matched to something happening on the paper.

The sheets below contain the complete addition, subtraction, and long-division programs, followed by the growing-gap printer from Chapter 2. Their shorter names, such as `Normalize`, stand for work also explained here. The arithmetic itself is the arithmetic of the chapters. What takes space is making every return to a place, every carried one, and every ending available to a worker who must follow the writing exactly.

A.1 Keeping a place in the addition

Write 587 and 468 on separate rows of squared paper, one digit in each square. Leave blank squares on both sides. Choose a clean row for the answer. Three place cards can now do the work of three fingers: `A` points to the seven, `B` to the eight, and `W` to the square where five will go. A fourth, `C`, points to a separate square for the carry. That square starts blank.

In Chapter 1, we left each carried one above its column. These sheets make a small economy: they reuse the same carry square as they travel left. Its mark always belongs to the column about to be worked. After seven plus eight, it holds one; after the next column, it holds the new one. None of the original digits needs to be crossed out.

An address on a place card consists of three numbers: page, column, row. Thus $(3, 5, 2)$ means page three, column five, row

two. Columns increase to the right and rows upward; negative numbers carry the numbering leftward or downward past zero. More joined paper extends a page, while another page provides a separate working surface. The name `A` stays on its card even when its address changes.

The instruction `Write(W,5)` puts five in the answer square. `Write(C,1)` records the carry. To reach the next column, `Move(A,0,-1,0)` changes the address on `A`: zero pages, one column left, zero rows. If it began at $(3, 5, 2)$, it now says $(3, 4, 2)$. The seven stays where it was. The three shifts in a `Move` instruction are always fixed numbers written on the sheet; `Move(FRESH,1,0,0)`, used later, takes the same column and row on the next page.

Before a card goes walking, another can save its place. `Place(U,A)` copies the address from `A` to `U`. Moving `A` afterward leaves `U` alone. `Place(A,U)` brings the first card back. Copying an address does not copy the digit at that address: it saves where to look.

The choice of what to write begins with a look. In `If(A,7) Goto foundseven`, a seven sends the worker to the line labeled `foundseven`. Any other mark sends the worker to the following line. A label is an ordinary name for a line, and `Goto` says where to continue reading. Apart from such jumps, the lines are followed in order. `Halt` ends the work. The sheet has finitely many lines and card names; the paper can grow as the work requires. Cards whose starting places are not supplied begin at $(0, 0, 0)$, until an instruction gives them a job.

A.2 What the tables say

For our seven and eight, look in the addition table with no incoming carry: row seven, column eight. Its entry is 5 1. The first mark goes in the answer square; the second goes in the carry square. At the next column, the one already there selects the other addition table. Row eight, column six gives 5 1 again.

The square symbol □ means blank paper. It has a row and column of its own in each table, because one number can run out before the other. There are eleven observations to allow for: ten

digits and blank. An entry such as 5□ writes five and erases the carry square. In the instructions, erasing is `Write(C,blank)`.

Both number squares are blank only when we have passed both ends. The bottom-right entry then includes a letter: *H* ends the addition columns, *Y* ends a successful subtraction, and *N* ends an unsuccessful one. Write the two indicated marks before taking that exit. For addition with a remaining carry, this writes the final one of 1055. For subtraction, it writes blank beyond the answer and clears the borrow square; the letter preserves the news of whether a borrow remained. No further column is worked.

Addition: incoming carry □

	0	1	2	3	4	5	6	7	8	9	□
0	0□	1□	2□	3□	4□	5□	6□	7□	8□	9□	0□
1	1□	2□	3□	4□	5□	6□	7□	8□	9□	01	1□
2	2□	3□	4□	5□	6□	7□	8□	9□	01	11	2□
3	3□	4□	5□	6□	7□	8□	9□	01	11	21	3□
4	4□	5□	6□	7□	8□	9□	01	11	21	31	4□
5	5□	6□	7□	8□	9□	01	11	21	31	41	5□
6	6□	7□	8□	9□	01	11	21	31	41	51	6□
7	7□	8□	9□	01	11	21	31	41	51	61	7□
8	8□	9□	01	11	21	31	41	51	61	71	8□
9	9□	01	11	21	31	41	51	61	71	81	9□
□	0□	1□	2□	3□	4□	5□	6□	7□	8□	9□	□□H

Addition: incoming carry 1

	0	1	2	3	4	5	6	7	8	9	□
0	1□	2□	3□	4□	5□	6□	7□	8□	9□	01	1□
1	2□	3□	4□	5□	6□	7□	8□	9□	01	11	2□
2	3□	4□	5□	6□	7□	8□	9□	01	11	21	3□
3	4□	5□	6□	7□	8□	9□	01	11	21	31	4□
4	5□	6□	7□	8□	9□	01	11	21	31	41	5□
5	6□	7□	8□	9□	01	11	21	31	41	51	6□
6	7□	8□	9□	01	11	21	31	41	51	61	7□
7	8□	9□	01	11	21	31	41	51	61	71	8□
8	9□	01	11	21	31	41	51	61	71	81	9□
9	01	11	21	31	41	51	61	71	81	91	01
□	1□	2□	3□	4□	5□	6□	7□	8□	9□	01	1□H

Subtraction: incoming borrow □

	0	1	2	3	4	5	6	7	8	9	□
0	0□	91	81	71	61	51	41	31	21	11	0□
1	1□	0□	91	81	71	61	51	41	31	21	1□
2	2□	1□	0□	91	81	71	61	51	41	31	2□
3	3□	2□	1□	0□	91	81	71	61	51	41	3□
4	4□	3□	2□	1□	0□	91	81	71	61	51	4□
5	5□	4□	3□	2□	1□	0□	91	81	71	61	5□
6	6□	5□	4□	3□	2□	1□	0□	91	81	71	6□
7	7□	6□	5□	4□	3□	2□	1□	0□	91	81	7□
8	8□	7□	6□	5□	4□	3□	2□	1□	0□	91	8□
9	9□	8□	7□	6□	5□	4□	3□	2□	1□	0□	9□
□	0□	91	81	71	61	51	41	31	21	11	□□Y

Subtraction: incoming borrow 1

	0	1	2	3	4	5	6	7	8	9	□
0	91	81	71	61	51	41	31	21	11	01	91
1	0□	91	81	71	61	51	41	31	21	11	0□
2	1□	0□	91	81	71	61	51	41	31	21	1□
3	2□	1□	0□	91	81	71	61	51	41	31	2□
4	3□	2□	1□	0□	91	81	71	61	51	41	3□
5	4□	3□	2□	1□	0□	91	81	71	61	51	4□
6	5□	4□	3□	2□	1□	0□	91	81	71	61	5□
7	6□	5□	4□	3□	2□	1□	0□	91	81	71	6□
8	7□	6□	5□	4□	3□	2□	1□	0□	91	81	7□
9	8□	7□	6□	5□	4□	3□	2□	1□	0□	91	8□
□	91	81	71	61	51	41	31	21	11	01	□□N

The compact instruction

```
0 Table(add,A,B,C,W,again,finish,finish)
```

asks for precisely this lookup. `A` and `B` give the two number squares, `C` gives the incoming carry, and `W` gives the answer square. Write the entry's first mark at `W` and its second at `C`. The last three names tell us where an ordinary column, a successful ending, and an unsuccessful ending lead. Addition gives both endings the same destination. Subtraction keeps them apart:

```
0  Table(sub,A,B,C,W,again,yes,no)
```

A table lookup can itself be written as instructions. Test the carry square for blank or one. Within either choice, test the upper square for each of its eleven possibilities, then the lower square for each of its eleven. The seven-and-eight choice with no carry ends:

```
0  Write(W,5)
1  Write(C,1)
2  Goto again
```

The two-blanks choice with a borrow still owed ends:

```
0  Write(W,blank)
1  Write(C,blank)
2  Goto no
```

Every other cell supplies its own two writes and jump in just this way. Two choices of carry or borrow, eleven upper observations, and eleven lower observations give $2 \times 11 \times 11 = 242$ choices. The full tables above supply all of them. A mark outside the stated choices leads to a halt for invalid arithmetic input.

A.3 Finishing the written answer

The columns of $1234 - 1234$ leave 0000. The subtraction has succeeded, but its answer needs tidying. Save the rightmost zero's place on a card called `UNIT`, and begin another card, `LEFT`, just before the answer. Walk right, passing blanks and erasing zeros, until a nonzero digit or the saved units place is reached. This walk leaves the last zero alone.

The place comparison is written `IfSame(LEFT,UNIT) Goto last`. It compares all three address numbers, rather than the marks on the paper. Equal addresses send us to `last`; otherwise the following line comes next. Here is the complete walk, called `Normalize`:

```
scan: IfSame(LEFT,UNIT) Goto last
If(LEFT,blank) Goto skip
If(LEFT,0) Goto erase
Goto done
erase: Write(LEFT,blank)
skip: Move(LEFT,0,1,0)
Goto scan
last: If(LEFT,blank) Goto zero
Goto done
zero: Write(LEFT,0)
done: Halt
```

On 0000, the first three zeros disappear and the last survives. On 00107, the walk erases two zeros and stops at one; the zero inside 107 is never approached. `LEFT` now identifies the first needed digit. The answer ends at `UNIT`, and its digits stay in their original squares. The last two lines also supply a zero if the units square itself is blank.

`Call Normalize` on a longer sheet means to do this walk, then resume at the line after the call. `Call Subtract` will mean the same thing for the subtraction sheet below. These are short names for already written work. To make one sheet without calls, insert a fresh copy of the named block at each call, rename its internal labels to keep them distinct, and replace each `Halt` in that copy with a jump back to the next line. Division uses subtraction, and subtraction uses normalization. Following that chain reaches the end after finitely many insertions.

A.4 The addition sheet

Return to 587 + 468. Save the units places of the two numbers on `AU` and `BU`, and the intended units place of the answer on `OU`. The last letter stands for units. The moving cards `A`, `B`, and `W` will leave these saved places behind as they travel left. Keep the carry square off all three rows.

To find a units place from the beginning of a number, walk right until the first blank and step back once. Starting at the five of 587

takes us past eight, seven, and blank, then back to seven. With `FIRST` at the first digit, the `FindUnits` sheet is:

```
Place(UNIT,FIRST)
seek: If(UNIT,blank) Goto back
Move(UNIT,0,1,0)
Goto seek
back: Move(UNIT,0,-1,0)
Halt
```

Save the result with `Place(AU,UNIT)`. Do the same for the other number and `BU`. Each supplied number must be a nonempty row of digits without gaps, with blank paper on either side. The two rows may be on different pages: their saved units places still tell us how to work through corresponding columns.

Here is the addition itself:

```
Place(A,AU)
Place(B,BU)
Place(W,OU)
Place(UNIT,OU)
Write(C,blank)
column: Table(add,A,B,C,W,again,finish,finish)
again: Move(A,0,-1,0)
Move(B,0,-1,0)
Move(W,0,-1,0)
Goto column
finish: Place(LEFT,W)
Call Normalize
Halt
```

The opening lines bring the working cards to their saved places, save the answer's units place, and clear the carry. At `column`, seven and eight write five and one. `again` moves the three working cards left and returns to the lookup. Eight and six, with the carry, write five and one; five and four, with the carry, write zero and one. One more move finds the two blanks. Their entry writes the remaining one and goes to `finish`. Normalization stops at that one immediately. Between `LEFT` and `UNIT` lies 1055.

A.5 The subtraction sheet

For 532 − 278, begin at two and eight. Borrowing gives twelve minus eight, so write four and record one in the borrow square. At the next column, three owes that extra one: subtract eight altogether, borrow, and write five. At the hundreds, five minus two minus the owed one gives two. The borrow clears. The next two blanks say yes, and the answer reads 254.

Use the same saved units cards and a fresh answer row. A separate square, `STATUS`, will hold `Y` or `N`. Keep it, like the borrow square `C`, away from both number rows and the answer row.

```
Place(A,AU)
Place(B,BU)
Place(W,OU)
Place(UNIT,OU)
Write(C,blank)
column: Table(sub,A,B,C,W,again,yes,no)
again: Move(A,0,-1,0)
Move(B,0,-1,0)
Move(W,0,-1,0)
Goto column
yes: Place(LEFT,W)
Call Normalize
Write(STATUS,Y)
Halt
no: Write(STATUS,N)
Halt
```

Now try 6 − 8. The first column writes eight and records a borrow. The next column finds two blanks with that borrow still owed, so the table sends us to `no`. The sheet writes `N` at `STATUS` and stops. Its scratch eight is unused; neither original number has changed. The usual signed answer is −2, but division needs the refusal: another eight will not fit.

Equality takes the other exit. 1234 − 1234 writes its four zeros, reaches two blanks without a borrow, and goes to `yes`. Normalization keeps one zero, and `STATUS` becomes `Y`. A successful answer always runs from `LEFT` through `UNIT`. A failed trial leaves no ac-

cepted answer. Both sheets finish because every return to `column` moves left through finite numbers; eventually both have ended.

A.6 The long-division sheet

The division 123/8 begins with remainder zero. Bring down one: no eight fits. Bring down two beside that one, making twelve: one subtraction leaves four, and a second fails. Write one in the quotient. Bring down three beside four, making forty-three: five subtractions leave three, so write five. The numerator has ended. Write the point and begin bringing down zeros.

These few sentences conceal several places to remember. `N` follows the numerator, while `NU` saves its last digit. `DL` and `DU` save the divisor's first and last digits. `Q` marks the next quotient square. For the accepted remainder, `RL` and `RU` hold the leftmost and units places. The next unused trial page has its units square saved on `FRESH`.

One concrete arrangement puts the numerator on page zero, row zero, and the divisor on page zero, row two, each starting at column zero. The quotient begins at column zero, row zero of page one. The initial remainder zero goes at $(2, 0, 0)$, named by `RU`; the first trial uses $(3, 0, 0)$, named by `FRESH`. Each later trial takes the next page. `FindUnits` supplies `NU` and `DU` from the supplied first places.

Six separate reminder squares occupy column zero, rows two through seven, of page one. They are `COUNT`, `C`, `STATUS`, `PHASE`, `PRINTED`, and `DONE`. The first counts successful trials; the next two belong to subtraction. The remaining three answer questions that affect the writing of the quotient. Has the decimal point appeared? Has the integer answer begun? Has the last numerator digit come down? Each starts blank and becomes one when its answer becomes yes.

Two short tasks finish the preparations. `Copy(A,B)` looks at the digit or blank at `A` and writes that same mark at `B`, leaving both addresses fixed. Eleven tests cover its eleven possible observations; the seven test writes seven, the blank test erases, and each then jumps past the remaining tests. This differs from `Place`, which

copies an address. `Increment(COUNT)` increases a digit by one: test zero and write one, test one and write two, and continue up to the last case, which tests eight and writes nine. Each of these nine choices then resumes the next instruction. Any other mark halts. A division round will never ask this little sheet to increase nine.

At `trial` below, subtraction gets the accepted remainder, the divisor, and a fresh answer page. Success moves the remainder cards to the new difference and increases the count. Failure jumps straight to `settled`, leaving those cards where they were. The sheet begins by checking the divisor. A moving card, `CHECK`, copies its first place from `DL` and walks through its digits looking for a nonzero one. If every digit is zero, the sheet writes `Z` at `STATUS` and halts.

```
Place(CHECK,DL)
check: If(CHECK,0) Goto checknext
Goto start
checknext: IfSame(CHECK,DU) Goto baddivisor
Move(CHECK,0,1,0)
Goto check
baddivisor: Write(STATUS,Z)
Halt
start: Write(RU,0)
Place(RL,RU)
Write(PHASE,blank)
Write(PRINTED,blank)
Write(DONE,blank)
read: Move(RU,0,1,0)
Copy(N,RU)
IfSame(N,NU) Goto lastinput
Move(N,0,1,0)
Goto prepare
lastinput: Write(DONE,1)
prepare: Place(LEFT,RL)
Place(UNIT,RU)
Call Normalize
Place(RL,LEFT)
Write(COUNT,0)
trial: Place(AU,RU)
Place(BU,DU)
```

```
Place(OU,FRESH)
Move(FRESH,1,0,0)
Call Subtract
If(STATUS,N) Goto settled
Place(RL,LEFT)
Place(RU,UNIT)
Increment(COUNT)
Goto trial
settled: If(PHASE,1) Goto fractionalanswer
If(PRINTED,1) Goto integeranswer
If(COUNT,0) Goto nextinput
Write(PRINTED,1)
integeranswer: Copy(COUNT,Q)
Move(Q,0,1,0)
nextinput: If(DONE,1) Goto point
Goto read
point: If(PRINTED,1) Goto putpoint
Write(Q,0)
Move(Q,0,1,0)
putpoint: Write(Q,dot)
Move(Q,0,1,0)
Write(PHASE,1)
Goto bringzero
fractionalanswer: Copy(COUNT,Q)
Move(Q,0,1,0)
bringzero: Move(RU,0,1,0)
Write(RU,0)
Goto prepare
```

Follow the first round after the point, starting with remainder three. `bringzero` writes zero to its right, making thirty. `prepare` tidies the remainder if needed and resets the count. The trials now make $30 - 8 = 22$, $22 - 8 = 14$, and $14 - 8 = 6$. Each success gives the remainder a new address. $6 - 8$ fails on yet another page. The accepted six and count three survive, so `fractionalanswer` copies three into the waiting quotient square. The next zero is appended to six, making sixty. Seven successes leave four; the following round gives five successes and zero. The display has reached 15.375.

Before the point, `PRINTED` lets the initial unsuccessful division of one by eight pass without writing an unwanted leading zero. If

the whole integer answer is zero, as in 1/4, `point` writes that one necessary zero before the point. Afterward, zeros have their full place value and every settled digit is written. The sheet therefore continues with 15.375000 . . ., or 0.250000 . . . for 1/4.

Why can one counting square suffice? With divisor eight, the remainder before a new digit comes down is at most seven. Even bringing down nine makes only seventy-nine, less than ten eights. In general, a remainder smaller than a positive whole-number divisor is at least one smaller. Multiplying it by ten leaves room for ten more; adding at most nine still falls short of ten divisors. At most nine trials succeed, and the next fails. Each trial crosses finitely many columns, so each round eventually writes its digit.

A.7 The growing-gap sheet

Keep the same answer card `Q`, but give the worker a different job. On another blank row, reserve a place called `BEGIN`. Put one tally there and save the square just after it as `END`. A moving card, `SCAN`, will cross the tallies, writing one answer zero for each. At the end it writes one, adds a tally, and starts crossing again.

```
Write(Q,0)
Move(Q,0,1,0)
Write(Q,dot)
Move(Q,0,1,0)
Place(END,BEGIN)
Write(END,X)
Move(END,0,1,0)
nextblock: Place(SCAN,BEGIN)
zeros: IfSame(SCAN,END) Goto one
Write(Q,0)
Move(Q,0,1,0)
Move(SCAN,0,1,0)
Goto zeros
one: Write(Q,1)
Move(Q,0,1,0)
Write(END,X)
Move(END,0,1,0)
Goto nextblock
```

The first four lines write 0.. With one tally, the crossing adds 01; with two, it adds 001; with three, it adds 0001. Thus the answer grows as

$$0.\underbrace{01}_{\text{one zero}}\ \underbrace{001}_{\text{two zeros}}\ \underbrace{0001}_{\text{three zeros}}\ \cdots .$$

`IfSame(SCAN,END)` ends each crossing when the moving card reaches the saved blank beyond the tallies. That distance is finite, however large it has become. The answer card moves right after every write, so the old digits remain untouched while the next block arrives.

Appendix B

A proof with the work left in

Two bags have something in common. We have not opened them, and nobody has told us what the shared object is. Even so, neither bag can be empty. That is the whole argument we will spread across the next few pages. Its conclusion is modest enough to let us watch what happens to every assumption along the way.

Call the bags b and c. As in Chapter 5, $n \in b$ says that the object called n is a member of b. The letter need not stand for a number. We could be talking about another bag. The sign $\wedge$ joins two statements with "and," so

$$(n \in b) \wedge (n \in c)$$

says that this same object belongs to both bags. Keeping the name the same matters. A red marble in one bag and a blue marble in the other would not establish that they have a member in common.

At present we do not have the object, or even a name for it. We have only the promise that one exists. Put that promise on paper:

$$\exists x \left((x \in b) \wedge (x \in c)\right).$$

Read $\exists x$ as "there is an object x such that." The rest specifies what it must do: belong to b and to c. Here x is a place in the sentence, waiting for an example. The sentence does not tell us which example fills it.

We can nevertheless give the unknown object a temporary name. Let n be that name, unused elsewhere in the argument. Open a branch of the proof and write

$$\big[(n \in b) \wedge (n \in c)\big]_1.$$

The brackets mark a temporary assumption. The small 1 is its label, so that we can later say exactly which assumption has finished its

job. We are investigating what follows from the promised example, whichever object it is. We have not reached into a bag and selected our favourite marble.

From an "and" statement we may take either part. This is one of the proof cards from Chapter 5, now doing an actual job:

$$\frac{(n \in b) \wedge (n \in c)}{n \in c}.$$

The statement above the bar is available; the statement below it is the part we keep. A worker checking the step compares the writing on the two sides of the bar. If the lower line said $c \in n$, the check would fail. Rearranging the furniture inside a formula can change who contains whom.

We could also take $n \in b$, then join the two pieces again. Combining information does no more, and no less, than preserve its parts:

$$\frac{n \in b \qquad n \in c}{(n \in b) \wedge (n \in c)}.$$

Each part still depends on assumption 1. Putting them on one line has not made that assumption disappear. A checker keeps track of these dependencies along with the statements themselves.

The shorter route to our conclusion is already open. Since $n \in c$, there is a member of c:

$$\frac{n \in c}{\exists y(y \in c)}.$$

This time we are giving an example for an existence claim. The name n appears above the bar, but the conclusion below does not mention it. We can close branch 1 and keep that conclusion. We began with a promise of a shared member and have derived a promise of a member of c, without finding out what either promise names. Taking the first part instead gives the same argument for b. Neither bag can be empty.

There are two permissions here, and they run in opposite directions. A known example lets us assert that an example exists. A bare existence claim lets us investigate an unknown example under a temporary name. The second permission does not deliver

an object to the desk. It allows an argument to proceed with the information already supplied.

Try carrying $n \in c$ out of the branch instead. Now the conclusion still uses n. Outside this branch, what does that name mean? We never assigned it a particular object there. The checker refuses the move. By contrast, "c has a member" makes no demand on the borrowed name. We can take that claim with us when the branch closes.

The name must be new when the branch opens, too. Suppose a marble already called n is known to be outside c. If we used its name for the promised common member, we would obtain both "n belongs to c" and "n does not belong to c." The shared member might have been a button all along. Our choice of a letter would have created the trouble. The worker can prevent this by inspecting the names in the starting claim, in the conclusion we intend to keep, and in the other assumptions still in use. The temporary name must be absent from all three.

Now take the longer route, through a contradiction. In addition to the shared-member promise, suppose e names an empty bag. We want to show that c is not e. Begin branch 1 afresh, again naming the unknown common member n. Open another branch inside it: try $c = e$. Equality lets us replace one name by the other. From $n \in c$ we therefore obtain $n \in e$.

But e was empty. In particular, n is not in it. Write this as $\neg(n \in e)$, using $\neg$ for "not." We have reached a statement and its negation. The contradiction rejects the equality we tried. All the work can remain on the sheet:

$\big[(n \in b) \wedge (n \in c)\big]_1$	name the promised member
$n \in c$	take the second part
$[c = e]_2$	try equality
$n \in e$	replace equal names
$\neg(n \in e)$	use emptiness
$\bot$	contradiction
$\neg(c = e)$	close assumption 2
$\neg(c = e)$	close branch 1.

The symbol $\perp$ marks the contradiction. Indentation shows which trials are still open. The worker follows the labels and the dependencies of the lines; the indentation helps our eyes.

The two final lines say the same thing. They do different work. The first closes the attempted equality while the unknown member still has its temporary name. The second closes the branch in which that name was introduced. Our conclusion mentions only c and e, so the temporary n can go. What remains is exactly the claim we wanted, resting on the shared-member promise and the emptiness of e.

The line $n \in e$ is still visible. We cannot reuse it as an established fact. It depended on the equality we rejected. Nor have we disproved the starting promise: an empty e can coexist quite peacefully with two other bags sharing a button. The contradiction tells us to reject the particular trial that produced it while the earlier assumptions remain in force. Naming that trial is part of the proof.

This is why a proof contains more information than a list of sentences. The lines must say where their authority comes from. An arithmetic worksheet keeps the borrowed one beside the column that needs it. Our proof sheet keeps a borrowed assumption beside the lines that need it. Moving either across the page without keeping its connection can manufacture a very persuasive wrong answer.

An unknown example is also different from an arbitrary object. Our n was unknown, but it was not unrestricted: the starting promise required it to belong to both bags. We cannot declare that every object belongs to c just because n does. We have followed one member under a temporary name, not investigated everything in the world.

For a genuinely arbitrary object, recall the two permissions of Adjunctive Set Theory. There is an empty collection, and we may add one object to a collection already available. Start with any object a. Take an empty collection and add a, obtaining a collection d. Then $a \in d$, so we can give d as an example for "there is a collection containing a," written $\exists y(a \in y)$. This claim no longer needs d or the name of the empty collection. Close the branches in which those names were introduced and keep the claim. We used

nothing special about a: it did not have to be a marble, an empty bag, or a member already promised to us. The argument therefore applies to every object. In symbols its conclusion is

$$\forall a \, \exists d \, (a \in d).$$

The sign $\forall a$ means "for every object a." Read the signs in their written order: every object has some collection containing it. We have not made one enormous collection containing everything. A different a may require a different d.

These little arguments are useful because we can see the errors as well as the successful moves. Reversing the names in membership changes the statement. Exporting the unknown member keeps a name we were allowed to borrow only temporarily. Generalizing from that member mistakes a restriction for freedom. None of these errors requires the checker to be offended by the conclusion. Each leaves a defect in the written argument.

Return finally to the worker who writes 7, moves one square right, and halts. A record of the work can show the blank starting square, the appearance of the 7, the change of the worker's place, and the halt. It must leave the written 7 where it was. Moving the worker does not give the digit permission to follow. Chapter 5 explains how the coding results let us express these checks in our set language and turn an actual completed run into a formal proof of its completion.

The existence claim then concerns a record rather than a member of a pictured bag. The logical moves are the ones we have just used. If a proof begins by assuming such a record exists, it may name it temporarily and reason about it. That does not execute the program. A claim that the worker has finished and a worker who has finished remain different things; the chapters about Henkin's models depend on keeping the difference visible.

The complete rule specifications and the coding results supporting that application are collected in the separate technical companion. Here the whole proof fits on a sheet. We can point to the place where the unknown member enters, to the equality that fails, and to the line that is allowed to survive it.

Sources and Further Reading

The workers, games, and mathematical worlds in these chapters have longer lives in the sources below. Some entries lead back to the original papers; others open up arguments that this book has kept close to the pencil and the decimal. The historical accounts also restore something an instruction sheet leaves out: the people who thought to write it.

The worker, self-reference, and formal proof

Sarah McLennan and Mary Gainer, "When the Computer Wore a Skirt: Langley's Computers, 1935–1970", NASA historical account. The first Langley computing pool belonged to NACA, NASA's predecessor. These professional computers performed skilled work, including checking calculations and developing methods. Our rule-following worker isolates one part of calculation; the expanding graph-paper desk is a mathematical model, not a reconstruction of their workplace.

Princeton's "An Early History of Computing at Princeton" reproduces Max Newman's contemporary description of Turing's independent work and discusses his subsequent Princeton years. Church's students enter this book in the order needed by its mathematical argument, not in the order of their doctorates. Turing did not develop his 1936 account as a member of a joint project with Kleene and Henkin.

Alan M. Turing, "On Computable Numbers, with an Application to the Entscheidungsproblem," *Proceedings of the London Mathematical Society* 42 (1936–37), 230–265. Turing opens with the idea of calculating a real's decimal by finite means; his formal computing machines in Section 2 produce binary digits. He separates the lasting output from erasable working symbols and distinguishes a finite machine description from its endless output. Chapter 1

relates a worker using graph paper and recorded places to a tape machine. Original paper, digitized copy.

Jeffry L. Hirst, "Representations of Reals in Reverse Mathematics," *Bulletin of the Polish Academy of Sciences. Mathematics* 55 (2007), 303–316, doi:10.4064/ba55-4-2. Hirst compares rapidly converging rational approximations, decimal expansions and Dedekind cuts. Sections 2–3 distinguish conversion for an individual real from a uniform conversion procedure. The computable reals form the same class under the digit and effective-approximation descriptions, while a general algorithm converting arbitrary effective approximations into a settled decimal expansion is unavailable. The distinction concerns the representations and the information supplied to a procedure. Publisher's full text.

David Hilbert, "The Foundations of Mathematics" (1927), translated in Jean van Heijenoort, ed., *From Frege to Gödel*. Hilbert treats formulas as concrete objects and derivations as operations governed by explicit rules. The proof checker follows this approach using natural-deduction proof trees. David Hilbert and Wilhelm Ackermann, *Grundzüge der theoretischen Logik* (1928), is a historical source for the first-order decision problem.

Stephen C. Kleene, *Introduction to Metamathematics* (1952), develops the classical computability and recursion-theorem framework. Kleene's Princeton oral-history interview gives a first-person account of Church's role. Chapter 4 proves the self-description construction used here with explicitly distinguished program builders and execution.

Zlatan Damnjanovic, "Mutual Interpretability of Robinson Arithmetic and Adjunctive Set Theory with Extensionality," *Bulletin of Symbolic Logic* 23 (2017), 381–404. The result covers adjunctive set theory with or without extensionality. doi:10.1017/bsl.2017.30. This establishes the strength of the two-axiom theory Empty and Adjunction used in Chapter 5. The chapter's uniform coding of computation records also uses the sequence machinery described in the next entry.

Albert Visser, "Pairs, Sets and Sequences in First-Order Theories," *Archive for Mathematical Logic* 47 (2008), 299–326, doi:10.1007/s00153-008-0087-1. The interpreted sequence machinery supplies uni-

form coding for finite records. Visser's "The Arithmetics of a Theory," *Notre Dame Journal of Formal Logic* 56 (2015), 81–119, doi:10.1215/00294527-2835029, develops compatible interpreted arithmetic and sequence structures. These papers give the full interpretation constructions; Chapter 2 of the *Technical Companion* identifies the coding properties needed for computation records.

Category, games, and worlds

René Baire, *Sur les fonctions de variables réelles* (1899), digitized dissertation. Chapter 3 develops category through events in a game that forms an infinite decimal. Drawers collect digit strings with a given finite beginning. Both decimal names of a shared numerical endpoint remain available; an event concerning a real number includes every address of that number. The successive-escape proof yields uncountability as a special case of Baire's category argument.

Baire's "Nouvelle démonstration d'un théorème sur les fonctions discontinues", *Bulletin de la Société Mathématique de France* 28 (1900), 173–179, revisits the work of his thesis on limits of continuous functions. It supplies the historical setting for the discussion of continuity and exceptional sets. Our decimal game presents a consequence of the method; it is not an account of how Baire discovered it.

R. Daniel Mauldin, ed., *The Scottish Book: Mathematics from The Scottish Café, with Selected Problems from The New Scottish Book*, second edition (2015), doi:10.1007/978-3-319-22897-6. Problem 43 and its commentary record Mazur's interval game and Banach's answer. In the decimal game of Chapter 3 and Chapter 4 of the *Technical Companion*, players choose finite digit blocks of variable length. The category characterization applies to that rule of play.

Leon Henkin, "The Completeness of the First-Order Functional Calculus," *Journal of Symbolic Logic* 14 (1949), 159–166, doi:10.2307/2267044. His retrospective "The Discovery of My Completeness Proofs" (1996), doi:10.2307/421107, explains the influence of Church's teaching and writings. Chapter 6 gives the

countable model-existence construction and derives compactness from it.

Asaf Karagila, *Forcing & Symmetric Extensions*, lecture notes, especially the opening sections on names and generic models. These develop the formal forcing relation, forcing theorem, and preservation results used in the coda. The coda and Chapter 4 of the *Technical Companion* assume a countable transitive ground model and construct a Henkin-style quotient of names. The ground-model assumption is additional to consistency of adjunctive set theory.

D. A. Martin, *Set Theory*, Math 223B lecture notes (2005), pp. 25–26, Lemma 3.7. The regular-open construction gives the completed Boolean operations. Chapter 4 of the *Technical Companion* expresses its join operation through the test of density inside a drawer and works two examples.

The final construction

Joel David Hamkins, “Every function can be computable!”, presents an alternative proof based on a suggestion of Vadim Kosoy: a program searches for a certificate excluding a finite pattern of its own behavior and is prepared to realize that pattern. Hamkins states that proof for a theory interpreting Peano arithmetic. Our formulation instead isolates the finite-verification and record-coding hypotheses it uses, with the adjunctive interpretation route described in Chapter 2 of the *Technical Companion*; that route is a separate dependency. This is the certificate reversal adapted in Chapter 7. Hamkins’s “The universal algorithm: a new simple proof of Woodin’s theorem” explains the stronger universal enumeration phenomenon. See also *The Modal Logic of Arithmetic Potentialism and the Universal Algorithm*, arXiv:1801.04599.

Chapter 7 adapts certificate reversal to one common quotient record, using the stated verification hypotheses and compactness. Its model realizes every standard prefix of a prescribed decimal; it need not extend a preselected model. The coda explains why ordinary forcing preserves existing computations, unlike the variation of nonstandard records used in the finale.